Introduction

Contents

Welcome to the Software Gardening Almanack, an open-source handbook of applied guidance and tools for sustainable software development and maintenance.

The Software Gardening Almanack is for anyone who creates or maintains software. The Almanack instructs individuals - software developers, engineers, project leads, scientists, and beyond - on best nurturing practices to build software that stands the tests of time. If you've ever wondered why software grows fast only to decay just as quickly, or how to design systems that sustain long-term innovation, this is for you. This handbook provides pragmatic guidance and tools to cultivate resilient software practices. Whether you're tending legacy code or planting new ideas, ***jump in and start growing***! 🌱

Overview

The structure of the book includes the following *core chapters*:

1. The **Garden Lattice**: a chapter exploring the human-centric elements of software.
2. The **Software Forest**: a chapter examining code and its role in shaping software.

3. The **Verdant Sundial**: a chapter illustrating the evolution of people and code, and their impact on software sustainability.

In addition there are also *supplementary chapters* which help support or demonstrate the content found within the core chapters:

- The **Seed Bank**: an applied chapter which illustrates concepts from the Almanack core through Jupyter notebooks.
- The **Garden Circle**: a space where we openly document principles for developing and maintaining the book as well as document a related application programming interface (API) for the `almanack` Python package.

Using the Almanack

The Software Gardening Almanack project provides two primary components:

- The **Almanack handbook**: the content found here helps educate, demonstrate, and evolve the concepts of sustainable software development.
- The `almanack` **package**: is a Python package which implements the concepts of the book to help improve software sustainability by generating organized metrics and running linting checks on repositories.

The `almanack` package

Please see our (Seed Bank)[seed-bank-intro] **Software Gardening Almanack Example** for a quick demonstration of using the `almanack` Python package. The package may be installed using the following, for example:

```
# install from pypi
pip install almanack

# install directly from source
pip install git+https://github.com/software-gardening/almanack.git
```

Once installed, the Almanack can be used to analyze repositories for sustainable development practices. Output from the Almanack includes metrics which are defined through `metrics.yml` as a Python dictionary (JSON-compatible) record structure. Package API documentation may be found within the package API section of the book.

You can use the Almanack package as a command-line interface (CLI):

```
# generate a table of metrics based on a repository
almanack table path/to/repository
```

```
# perform linting-style checks on a repository
almanack check path/to/repository
```

We provide pre-commit hooks to enable you to run the Almanack as part of your automated checks for developing software projects. Add the following to your `pre-commit-config.yaml` in order to use the Almanack.

For example:

```
# include this in your pre-commit-config.yaml
- repo: https://github.com/software-gardening/almanack
    rev: v0.1.1
    hooks:
    - id: almanack-check
```

Inspiration

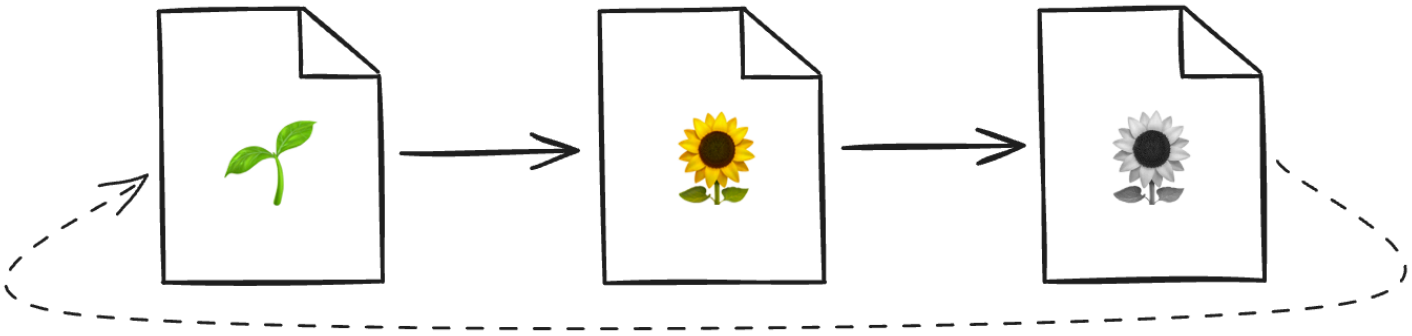


Fig. 1 Software is created, grows, and decays over time.

Software experiences development cycles, which accumulate errors over time. However, these cycles are not well understood nor are they explicitly cultivated with the impacts of time in mind. Why does software grow quickly only to decay just as fast? How do software bugs seem to appear in unlikely scenarios?

These cycles follow patterns from life: software is created, grows, decays, and so on (sometimes in surprising or seemingly unpredictable ways). Software is also connected within a complex system of relationships (similar to the complex ecology of a garden). The Software Gardening Almanack posits we can understand these software lifecycle patterns and complex relationships in order to build tools which sustain or maintain their development long-term.

> *"The 'planetary garden' is a means of considering ecology as the integration of humanity – the gardeners – into its smallest spaces. Its guiding philosophy is based on the principle of the 'garden in motion': do the most **for**, the minimum **against**."* - Gilles Clément

The content within the Software Gardening Almanack is inspired by ecological systems (for example, as in *planetary gardening*). We also are galvanized by the *scientific method*, and *almanacks* (or earlier *menologia rustica*) in documenting, expecting, and optimizing how we plan for time-based changes in agriculture (among other practices and traditions).

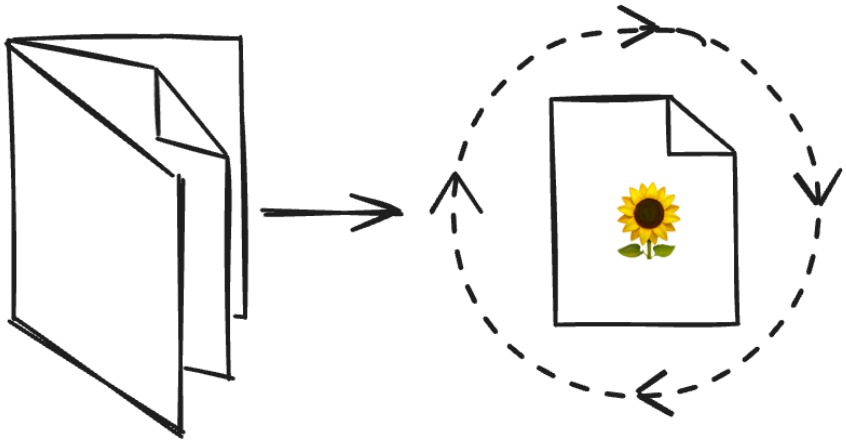


Fig. 2 Almanacks help us understand and influence the impacts of time on the things we grow.

The Software Gardening Almanack helps share knowledge on how to cultivate change in order to nurture software (and software practitioners) for long periods of time. We aspire to define, practice, and continually improve a craft of *software gardening* to nourish existing or yet-to-be software projects and embrace a more resilient future.

Motivation

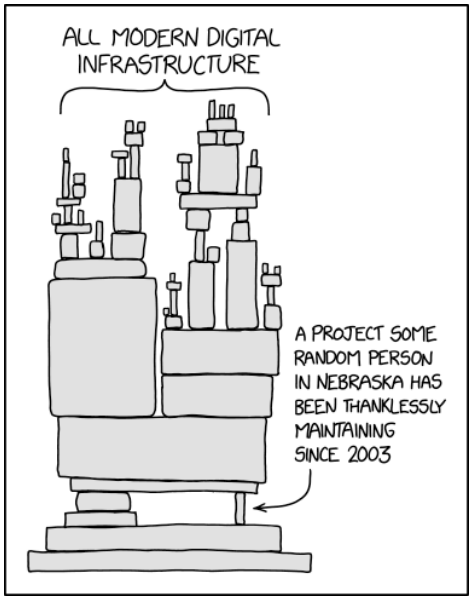


Fig. 3 We depend on a delicate network of software which changes over time.

(Image source: 'Dependency' comic by Randall Munroe, XKCD)

Our world is surrounded by systems of software which enable and enhance life. These systems are both invisible and sometimes brittle, breaking in surprising ways (for example, beneath uneven pressures as in the above XKCD comic). At the same time, loose coupling of these software systems also forms the basis of innovation through multidimensional, emergent growth patterns. We seek to embrace these software systems which are innately in motion, embracing diversity without destroying it to perpetuate discovery and enhance future life.

> *"Start where you are. Use what you have. Do what you can."* - Arthur Ashe

We suggest reading or using this content however it best makes sense to a reader. Just as with gardening, sometimes the best thing to do is jump in! We structure each section in a modular fashion, providing insights with cited prerequisites. We provide links and other reference materials for further reading where beneficial. Please let us know how we can improve by opening GitHub issues or creating new discussion board posts (see our `contributing.md` guide for more information)!

Who's this for?

> *"We'll share stories from the heart. All are welcome here."* - Alexandra Penfold

The Software Gardening Almanack is designed for developers of any kind. When we say *developers* here we mean anyone engaging in pursuing their vision towards a goal using software (including scientists, engineers, project leads, managers, etc). The content will engage readers with pragmatic ideas, keeping the barrier to entry low.

Acknowledgements

This work was supported by the Better Scientific Software Fellowship Program, a collaborative effort of the U.S. Department of Energy (DOE), Office of Advanced Scientific Research via ANL under Contract DE-AC02-06CH11357 and the National Nuclear Security Administration Advanced Simulation and Computing Program via LLNL under Contract DE-AC52-07NA27344; and by the National Science Foundation (NSF) via SHI under Grant No. 2327079.

Garden Lattice

Fig. 4 The garden lattice facilitates growth on our software gardening journeys. (Image source: GL1).

> “To plant a garden is to believe in tomorrow.” - Audrey Hepburn

We enter the book through a garden lattice, both an acknowledgement and forward recognition of people who cultivate software. The garden lattice is a celebration of our individual vibrancy and connection to the world around us. “Here you will give your gifts and meet your responsibilities.” GL2 It also is a place to view our collective fates; as we grow an understanding emerges that we must commit ourselves toward a sustainable future through tending the garden. This sustainability is undeniably coupled to our natural relationships, persisted even in the software realm.

Early roots from Ada Lovelace’s work on an analytical engine algorithm in 1842 provide a glimpse of the natural world’s influence on software development: “… the Analytical Engine weaves algebraical patterns just as the Jacquard-loom weaves flowers and leaves.” GL3 Many years later, software development remains an effort of intertwining beauty and function to enhance well-being through a growing system, or lattice, of interconnected parts built upon layers of personal and technical advances.

Lattices, reminiscent of living structures built with willow or bamboo, emerge as a weaved foundation, both flexible and systematically resilient, protecting and inspiring new growth. “The willow submits to the wind and prospers until one day it is many willows - a wall against the wind.” [GL4]. These lattices also signify our collective friendship and interdependence. They echo the sentiment of “Be like bamboo. The higher you grow, the deeper you bow,” (a Chinese proverb) embodying strength through flexibility and unity. This concept mirrors the organic growth seen in software gardens, where a system of interweaved latticework supports the development of intricate and resilient structures.

The lattice also guards against both direct and indirect challenges we face on our journey as software gardeners. These encumbrances can “overheat” our growth potential, leading us to early burnout or burn-down of gardens. The lattice helps us as a carbon sequester, mitigating the excess heat caused by an imbalance in emissions, similar to bamboo garden forests [GL5]. Our individual experiences and personal network consume these difficulties to further grow the lattice through acts of harmonization (such as collaboration or apprenticeship). We care for these experiences together on the lattice because “all flourishing is mutual.” [GL2].

[GL1] Benjamin Gimmel. Round window with lattice. 2005. URL: https://commons.wikimedia.org/wiki/File:Rundes_Fenster_mit_Gitter.jpg (visited on 2024-05-17).

[GL2](1,2) Robin Wall Kimmerer. *Braiding sweetgrass: indigenous wisdom, scientific knowledge and the teachings of plants*. Milkweed Editions, Minneapolis, Minn, first paperback edition edition, 2013. ISBN 978-1-57131-356-0.

[GL3] J. Fuegi and J. Francis. Lovelace & babbage and the creation of the 1843 'notes'. *IEEE Annals of the History of Computing*, 25(4):16–26, October 2003. URL: https://ieeexplore.ieee.org/document/1253887/ (visited on 2024-05-10), doi:10.1109/MAHC.2003.1253887.

[GL4] Frank Herbert and Brian Herbert. *Dune*. The Dune chronicles. Ace, New York, ace premium edition edition, 2010. ISBN 978-0-441-17271-9.

[GL5] Arun Jyoti Nath, Rattan Lal, and Ashesh Kumar Das. Managing woody bamboos for carbon farming and carbon trading. *Global Ecology and Conservation*, 3:654–663, January 2015. URL: https://linkinghub.elsevier.com/retrieve/pii/S2351989415000281 (visited on 2024-05-12), doi:10.1016/j.gecco.2015.03.002.

Understanding

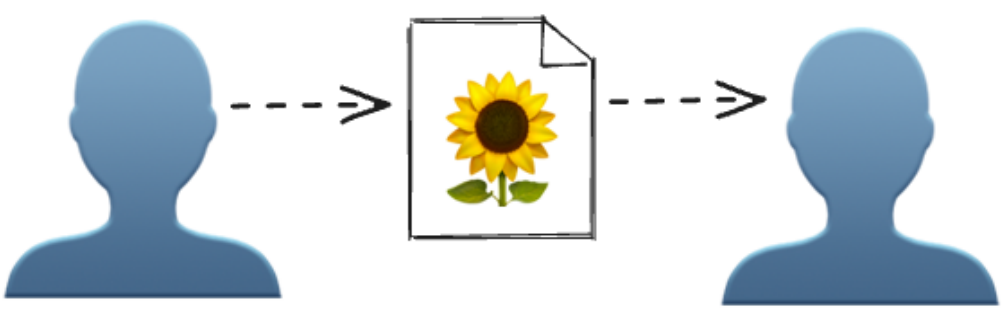


Fig. 5 We transfer understanding of software projects in the form of documentation artifacts.

Shared understanding forms the heartbeat of any successful software project. Peter Naur, in his work "Programming as Theory Building" (1985), emphasized that programming is fundamentally about building a shared theory of the project among all contributors [GLU1]. Understanding encompasses not only the technical aspects of software, but also the collaborative and ethical dimensions that ensure a software project's growth and sustainability. Just as a gardener must understand the needs of each plant to cultivate a thriving garden, contributors to a project must grasp its goals, structure, and community standards to foster a productive and harmonious environment. Documentation, of which there are many different forms that we describe below, cultivates a shared understanding of a software project.

Common files for shared understanding

In software development, certain files are expected to ensure project scope, clarity, collaboration, and legal compliance. These files serve as the foundational elements that guide contributors and users, much like a well-tended garden benefits from clear paths and signs. When present, these files are also correlated with increased rates of project success [GLU2].

Historically, developers had written the following files in plain-text without formatting. More recently, developers prefer markdown to format the documents with rich styles and media. GitHub, GitLab, and other software source control hosting platforms often automatically render markdown materials in HTML.

README

The `README` file is the cornerstone of any repository, providing essential information about the project. Its presence signifies a well-documented and accessible project, inviting contributors and users alike to understand and engage with the work. A well-structured `README` will reference other pieces of important information that exist elsewhere, such as dependencies and other files we describe below. A repository thrives when its purpose and usage are communicated through a `README` file, much like a garden benefits from a clear plan and understanding of its layout.

> "A good rule of thumb is to assume that the information contained within the README will be the only documentation your users read. For this reason, your README should include how to install and configure your software, where to find its full documentation, under what license it's released, how to test it to ensure functionality, and acknowledgments." [GLU3]

For further reading on `README` file practices, see the following resources:

- The Turing Way: Landing Page - README File

- GitHub: About `README`s
- Makeareadme.com

CONTRIBUTING

A `CONTRIBUTING` file outlines guidelines for how to add to the project. Its presence fosters a welcoming environment for new contributors, ensuring that they have the necessary information to participate effectively. In the same way that a garden flourishes when there's good understanding of how to provide care to the garden and gardeners, a project grows stronger when contributors are guided by clear and inclusive instructions. The resulting nurturing environment fosters growth, resilience, and a sense of community, ensuring the project remains healthy and vibrant.

The `CONTRIBUTING` file should outline all development procedures that are specific to the project. For example, the file might contain testing guidelines, linting/formatting expectations, release cadence, and more. Consider writing this file from the perspective of both an outsider and a beginner: an outsider who might enhance your project by adding a new task or completing an existing task, and a beginner who seeks to understand your project from the ground up [GLU4].

For further reading on `CONTRIBUTING` file practices, see the following resources:

- GitHub: Setting guidelines for repository contributors
- Mozilla: Wrangling Web Contributions: How to Build a CONTRIBUTING.md

CODE_OF_CONDUCT

The `CODE_OF_CONDUCT` (CoC) file sets the standards for behavior within the project community. Its presence signals a commitment to maintaining a respectful and inclusive environment for all participants. It also may provide an outline for acceptable participation when it comes to conflicts of interest. A CoC is a foundational document that defines community values, guides governance and moderation, and signals inclusivity within the project, particularly for vulnerable contributors. [GLU5] A project community benefits from a shared understanding of respectful and supportive interactions, ensuring that all members can contribute positively, much like a garden requires a harmonious ecosystem to thrive.

For further reading and examples of `CODE_OF_CONDUCT` files, see the following:

- Contributor Covenant Code of Conduct
- Example: Python Software Foundation Code of Conduct
- Example: Rust Code of Conduct

LICENSE

A `LICENSE` file specifies the terms under which the project's code can be used, modified, and shared.

> "When you make a creative work (which includes code), the work is under exclusive copyright by default. Unless you include a license that specifies otherwise, nobody else can copy, distribute, or modify your work without being at risk of take-downs, shake-downs, or litigation. Once the work has other contributors (each a copyright holder), "nobody" starts including you." GLU6. The presence of a `LICENSE` file is crucial for legal clarity, encourages the responsible use or distribution of the project, and is considered an indicator of project maturity GLU7.

Just as a garden's health depends on understanding natural laws and respecting boundaries, a project's sustainability hinges on clear licensing that safeguards and empowers both creators and users, cultivating a culture of trust and collaboration.

For further reading and examples of `LICENSE` files, see the following:

- OSF: Licensing
- GitHub: Licensing a repository
- Choosealicense.com
- SPDX License List

Project documentation

Common documentation files like `README`'s, `CONTRIBUTING` guides, and `LICENSE` files are only a start towards more specific project information. Comprehensive project documentation is akin to a detailed gardener's notebook for a well-maintained project, illustrating how the project may be used and the guiding philosophy. This includes in-depth explanations of the project's architecture, practical usage examples, application programming interface (API) references, and development workflows. Oftentimes this type of documentation is provided through a "documentation website" or "docsite" to facilitate a user experience that includes a search bar, multimedia, and other HTML-enabled features.

Such documentation should strive to ensure that both novice and seasoned contributors can grasp the project's complexities and contribute effectively by delivering valuable information. Writing valuable content entails conveying information that the code alone cannot communicate to the user GLU8. In addition to increased value from understanding, software tends to be more extensively utilized when developers offer more comprehensive documentation GLU9. Just as a thriving garden benefits from meticulous care instructions and shared horticultural knowledge, a project flourishes when its documentation offers a clear and thorough guide to its inner workings, nurturing a collaborative and informed community.

Project documentation often exists within a dedicated `docs` directory where the materials may be created and versioned distinctly from other code. Oftentimes this material will leverage a specific documentation tooling

technology in alignment with the programming language(s) being used (for example, Python projects often leverage Sphinx). These tools often increase the utility of the output by styling the material with pre-built HTML themes, automating API documentation generation, and an ecosystem of plugins to help extend the capabilities of your documentation without writing new code.

For further reading and examples of deep Project documentation, see the following:

- Berkeley Library: How to Write Good Documentation
- Write the Docs: Software documentation guide
- Overcoming Open Source Project Entry Barriers with a Portal for Newcomers

[GLU1] Peter Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, May 1985. URL: https://linkinghub.elsevier.com/retrieve/pii/0165607485900328 (visited on 2025-01-02), doi:10.1016/0165-6074(85)90032-8.

[GLU2] Jailton Coelho and Marco Tulio Valente. Why modern open source projects fail. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 186–196. Paderborn Germany, August 2017. ACM. URL: https://dl.acm.org/doi/10.1145/3106237.3106246 (visited on 2024-12-31), doi:10.1145/3106237.3106246.

[GLU3] Benjamin D. Lee. Ten simple rules for documenting scientific software. *PLOS Computational Biology*, 14(12):e1006561, December 2018. URL: https://dx.plos.org/10.1371/journal.pcbi.1006561 (visited on 2024-12-31), doi:10.1371/journal.pcbi.1006561.

[GLU4] Christoph Treude, Marco A. Gerosa, and Igor Steinmacher. Towards the First Code Contribution: Processes and Information Needs. 2024. Version Number: 1. URL: https://arxiv.org/abs/2404.18677 (visited on 2024-12-31), doi:10.48550/ARXIV.2404.18677.

[GLU5] Hana Frluckaj and James Howison. Codes of Conduct in Open Source. In Daniela Damian, Kelly Blincoe, Denae Ford, Alexander Serebrenik, and Zainab Masood, editors, *Equity, Diversity, and Inclusion in Software Engineering*, pages 295–308. Apress, Berkeley, CA, 2024. URL: https://link.springer.com/10.1007/978-1-4842-9651-6_17 (visited on 2025-01-02), doi:10.1007/978-1-4842-9651-6_17.

[GLU6] Inc. GitHub. No license. Accessed: 2025-01-02. URL: https://choosealicense.com/no-permission/.

[GLU7] Deekshitha, Rena Bakhshi, Jason Maassen, Carlos Martinez Ortiz, Rob van Nieuwpoort, and Slinger Jansen. RSMM: A Framework to Assess Maturity of Research Software Project. June 2024. arXiv:2406.01788 [cs]. URL: http://arxiv.org/abs/2406.01788 (visited on 2024-10-02).

[GLU8] **missing author in henney_97_2010**

[GLU9] Awan Afiaz, Andrey Ivanov, John Chamberlin, David Hanauer, Candace Savonen, Mary J. Goldman, Martin Morgan, Michael Reich, Alexander Getka, Aaron Holmes, Sarthak Pati, Dan Knight, Paul C. Boutros, Spyridon Bakas, J. Gregory Caporaso, Guilherme Del Fiol, Harry Hochheiser, Brian Haas, Patrick D. Schloss, James A. Eddy, Jake Albrecht, Andrey Fedorov, Levi Waldron, Ava M. Hoffman, Richard L. Bradshaw, Jeffrey T. Leek, and Carrie Wright. Evaluation of software impact designed for biomedical research: Are we measuring what's meaningful? June 2023. arXiv:2306.03255 [cs, q-bio]. URL: http://arxiv.org/abs/2306.03255 (visited on 2024-10-02).

Software Forest

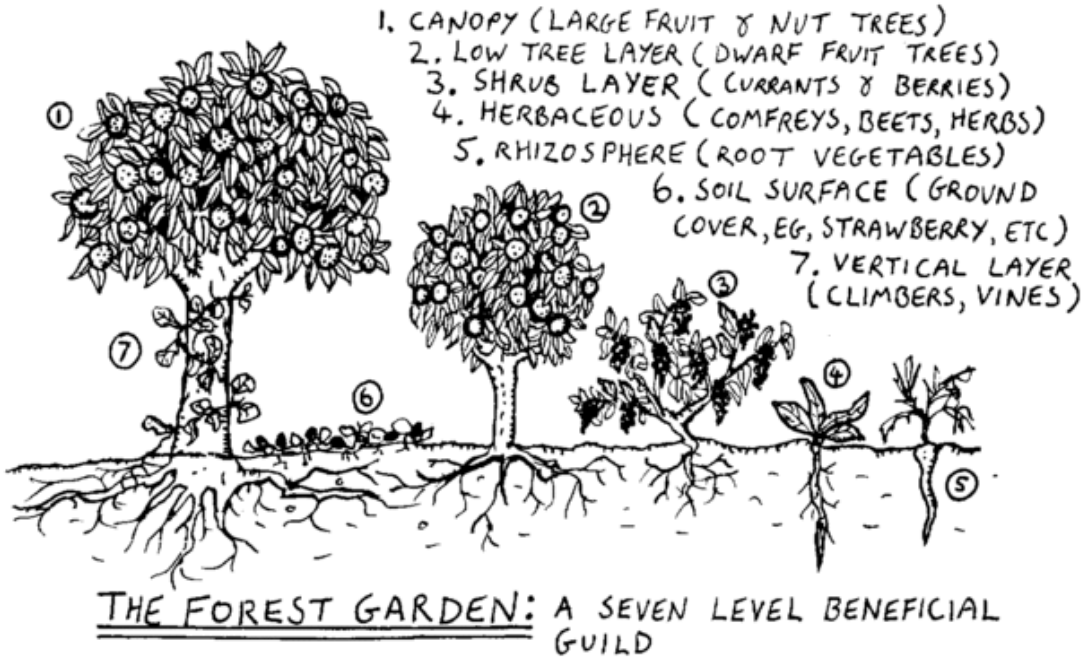


Fig. 6 The software forest is multi-layered like a forest garden. (Image source: SF1).

The software forest is a multidimensional garden, enriched by connections and minimal, iterative changes. Both the practice and artifacts of software development can seem daunting, like an unfamiliar forest full of mystery and surprises. However, through software gardening, we see these forests as opportunities for balanced cultivation.

> "Called Three-Dimensional Forestry or Forest Farming, … the three 'dimensions' of [Toyohiko Kagawa's] 3-D system were the trees as conservers of the soil and suppliers of food and the livestock which benefited from them." SF2, SF3

Forest gardening offers innovations from an ecological cultivation perspective by caring for trees, understory vegetation, soil, and other environmental factors. Forest gardens take time to create because their slow-growing nature requires patience and careful, long-term nurturing. Software forests are also created gradually, "… starting with the existing, living, breathing system, and working outward, a step at a time, in such a way as to not undermine the system's viability." SF4 Nourishing the system in this way helps us practice *primum non nocere* ("first, do no harm") to the garden and components within, conserving energy for necessary adaptations.

Dense networks of relationships surround software forest gardens in the form of software environments. Some of these environments are portable, similar to Wardian cases that provide loosely coupled habitats for software organisms (for example, through environment managers). Others are exhibited through abstract relationships as

nurse logs, *Hügelkultur*, or mycorrhizal-like associations which redistribute energy among the entirety of a software region.

As software gardeners we must consider how energy is transferred to various portions of the forest. Adventitious code, which “volunteers” by accident of growth, may create scenarios of energy waste in the form of cognitive or performance misalignment. All waste in the forest is food [SF5], therefore we must imagine and craft new homes for wasted energy through appropriate means. This chaos of this soft-scaping (curating the software garden towards a goal) is in constant flux, tugging on our capacities as student and teacher of the garden to encourage growth and avoid harm.

“What gardens demand of us is lifelong learning.” [SF6] We discover and are discovered within the software garden. As software forest gardens grow, they demonstrate emergent properties which are well suited for practices such as evolutionary architecture, object-orientated design, and coupling analysis. These assist software gardeners as a shed of multi-dimensional concepts that must be employed to truly achieve effective multi-layered software garden maintenance.

[SF1] Graham Burnett. Forest garden diagram to replace the one that currently exists on the article. 2006. URL: https://commons.wikimedia.org/wiki/File:Forgard2-003.gif (visited on 2024-05-17).

[SF2] Robert A. de J. Hart. *Forest gardening: cultivating an edible landscape*. Chelsea Green Pub. Co, White River Junction, Vt, 1996. ISBN 978-0-930031-84-8.

[SF3] Kagawa, T. and S. Fujisaki. Rittai nogyo no riron to jissen (theory and practice of three-dimensional structural farming). *Tokyo, Japan: Nihonhyoronsya*, 1935.

[SF4] Brian Foote and Joseph Yoder. Big ball of mud. *Pattern languages of program design*, 4:654–692, 1997.

[SF5] William McDonough. Waste equals food: our future and the making of things. *Awakening–The Upside of Y2k, The Printed Word, Spokane*, 1998.

[SF6] Viviane Stappmanns, Mateo Kries, Vitra Design Museum, and Wüstenrot Stiftung, editors. *Garden futures: designing with nature*. Vitra Design Museum, Weil am Rhein, 2023. ISBN 978-3-945852-53-8.

Verdant Sundial

Fig. 7 The verdant sundial casts shadows of past and future intention observed in the present. (Image source: VS1).

> “The present is the ever moving shadow that divides yesterday from tomorrow. In that lies hope.”
> - Frank Lloyd Wright

The software garden is intertwined in a story of time which is exhibited like a shadow on a moss-covered sundial. Time is observed indirectly through a kind of software archeology in past-tense material artifacts (files) and present-tense operation (procedures). These artifacts act as shadows of our prior creative intentions where procedures become the living present materialized promise of those intentions. “This brings an uncanny sense of living in a state where the future is a dimension of our present reality.” VS2 The future is elusive but present in software artifacts, similar to rings of a tree that depict the promise of continued stability or hope of change.

These software tree rings are vessels of temporal data which offer a unique insight into growth patterns. We can study these patterns like dendronchronology: “Peel away the hard, rough bark and there is a living document, history recorded in rings of wood cells.” VS3 This peeling helps us understand the chronological nature of software and how it evolves. It also shares patterns of temporal pulsing - not all times of the software garden are the same.

These software pulses give visibility into distinct growing or receding patterns. Some code exhibits annual behavior, requiring continual replanting efforts each season. Others are perennials, undergoing seasonal change

perpetuated by periods of vibrancy and dormancy. Seasons of software promote differing patterns among these. Some systems may prepare for winter (for instance, during resource scarcity, or periods of high performance stress) through adaptable conservation only to fervently bloom during their spring.

Seasonal chronology in software benefits also from an awareness of kairos, the opportune timing of our actions amidst time. As we gain experience in software gardens we begin to anticipate seasonal change which helps us project preparatory change. Each software gardening moment amidst this cascade of season includes a series of options of varying kairological fitness. Assessing these moments of their fit within patterns helps us gain wisdom as we contemplate time in software gardens; like annual flowers, how do we anticipate and accept the limited lifecycle of some software? Like mighty trees, how do we build software to last 100 years or more?

[VS1] Elfward. Garden sundial in an epping garden. 2015. URL: https://commons.wikimedia.org/wiki/File:Sundial_2916_HDR.jpg (visited on 2024-05-18).

[VS2] Timothy Morton. *Hyperobjects: Philosophy and Ecology after the End of the World*. University of Minnesota Press, Minneapolis, 2013. ISBN 9780816689231.

[VS3] Carolyn Gramling. 'tree story' explores what tree rings can tell us about the past. *Science News*, June 2020. Accessed: 2024-05-18. URL: https://www.sciencenews.org/article/tree-story-book-explores-what-tree-rings-can-tell-us-about-past.

Seed Bank

The seed bank is a section of the Software Garden Almanack associated with applied demonstrations of concepts and technologies associated with other areas of content.

Software Gardening Almanack Example

This notebook demonstrates installing and using the Software Gardening Almanack Python package. The Almanack is an open-source handbook of applied guidance and tools for sustainable software development and maintenance.

```
# install the almanack from pypi
import json

import pandas as pd

import almanack

!pip install almanack
```

```
Requirement already satisfied: almanack in /home/runner/.cache/pypoetry/virtualenvs/almanack-PE
Requirement already satisfied: charset-normalizer<4.0.0,>=3.4.0 in /home/runner/.cache/pypoetry
Requirement already satisfied: defusedxml<0.8.0,>=0.7.1 in /home/runner/.cache/pypoetry/virtual
Requirement already satisfied: fire<0.8,>=0.6 in /home/runner/.cache/pypoetry/virtualenvs/alman
Requirement already satisfied: pygit2<2.0.0,>=1.15.1 in /home/runner/.cache/pypoetry/virtualenv
Requirement already satisfied: pyyaml<7.0.0,>=6.0.1 in /home/runner/.cache/pypoetry/virtualenvs
Requirement already satisfied: requests<3.0.0,>=2.32.3 in /home/runner/.cache/pypoetry/virtuale
Requirement already satisfied: tabulate<0.10.0,>=0.9.0 in /home/runner/.cache/pypoetry/virtuale
Requirement already satisfied: termcolor in /home/runner/.cache/pypoetry/virtualenvs/almanack-P
Requirement already satisfied: cffi>=1.16.0 in /home/runner/.cache/pypoetry/virtualenvs/almanac
Requirement already satisfied: idna<4,>=2.5 in /home/runner/.cache/pypoetry/virtualenvs/almanac
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/runner/.cache/pypoetry/virtualenvs/a
Requirement already satisfied: certifi>=2017.4.17 in /home/runner/.cache/pypoetry/virtualenvs/a
Requirement already satisfied: pycparser in /home/runner/.cache/pypoetry/virtualenvs/almanack-P
```

```
# clone the almanack repo for example usage below
# we will apply the almanack tool to the almanack repo
!git clone https://github.com/software-gardening/almanack
```

```
fatal: destination path 'almanack' already exists and is not an empty directory.
```

```
%%time


# gather the almanack table using the almanack repo as a reference
almanack_table = almanack.metrics.data.get_table("almanack")

# show the almanack table as a Pandas DataFrame
pd.DataFrame(almanack_table)
```

```
CPU times: user 2.42 s, sys: 114 ms, total: 2.53 s
Wall time: 6.21 s
```

	name	id	result-type	sustainability_correlation	description	correction_guidance	
0	repo-path	SGA-META-0001	str	0	Repository path (local directory).	None	/hor
1	repo-commits	SGA-META-0002	int	0	Total number of commits for the repository.	None	
2	repo-file-count	SGA-META-0003	int	0	Total number of files tracked within the repos...	None	
3	repo-commit-time-range	SGA-META-0004	tuple	0	Starting commit and most recent commit for the...	None	
4	repo-days-of-development	SGA-META-0005	int	0	Integer representing the number of days of dev...	None	
5	repo-commits-per-day	SGA-META-0006	float	0	Floating point number which represents the num...	None	
6	almanack-table-datetime	SGA-META-0007	str	0	String representing the date when this table w...	None	
7	almanack-version	SGA-META-0008	str	0	String representing the version of the almanac...	None	
8	repo-primary-language	SGA-META-0009	str	0	Detected primary programming language of the r...	None	
9	repo-primary-license	SGA-META-0010	str	0	Detected primary license of the repository.	None	
10	repo-doi	SGA-META-0011	int	0	Repository DOI value detected from CITATION.cf...	None	
11	repo-doi-publication-	SGA-META-	str	0	Repository DOI	None	

	name	id	result-type	sustainability_correlation	description	correction_guidance	
	date	0012			publication date detected from ...		
12	repo-almanack-score	SGA-META-0013	dict	0	Dictionary of length three, including the foll...	None	{'a
13	repo-includes-readme	SGA-GL-0001	bool	1	Boolean value indicating the presence of a REA...	Consider adding a README file to the repository.	
14	repo-includes-contributing	SGA-GL-0002	bool	1	Boolean value indicating the presence of a CON...	Consider adding a CONTRIBUTING file to the rep...	
15	repo-includes-code-of-conduct	SGA-GL-0003	bool	1	Boolean value indicating the presence of a COD...	Consider adding a CODE_OF_CONDUCT file to the ...	
16	repo-includes-license	SGA-GL-0004	bool	1	Boolean value indicating the presence of a LIC...	Consider adding a LICENSE file to the repository.	
17	repo-is-citable	SGA-GL-0005	bool	1	Boolean value indicating the presence of a CIT...	Consider adding a CITATION file to the reposit...	
18	repo-default-branch-not-master	SGA-GL-0006	bool	1	Boolean value indicating that the repo uses a ...	Consider using a default source control branch...	
19	repo-includes-common-docs	SGA-GL-0007	bool	1	Boolean value indicating whether the repo incl...	Consider including project documentation throu...	
20	repo-unique-contributors	SGA-GL-0008	int	0	Count of unique contributors since the beginni...	None	
21	repo-unique-contributors-past-year	SGA-GL-0009	int	0	Count of unique contributors within the last y...	None	

	name	id	result-type	sustainability_correlation	description	correction_guidance
22	repo-unique-contributors-past-182-days	SGA-GL-0010	int	0	Count of unique contributors within the last 1...	None
23	repo-tags-count	SGA-GL-0011	int	0	Count of the number of tags within the reposit...	None
24	repo-tags-count-past-year	SGA-GL-0012	int	0	Count of the number of tags within the reposit...	None
25	repo-tags-count-past-182-days	SGA-GL-0013	int	0	Count of the number of tags within the reposit...	None
26	repo-stargazers-count	SGA-GL-0014	int	0	Count of the number of stargazers on repositor...	None
27	repo-uses-issues	SGA-GL-0015	bool	1	Whether the repository uses issues (for exampl...	Consider leveraging issues for tracking bugs a...
28	repo-issues-open-count	SGA-GL-0016	int	0	Count of open issues for repository.	None
29	repo-pull-requests-enabled	SGA-GL-0017	bool	1	Whether the repository enables pull requests.	Consider enabling pull requests for the reposi...
30	repo-forks-count	SGA-GL-0018	int	0	Count of forks of the repository.	None
31	repo-subscribers-count	SGA-GL-0019	int	0	Count of subscribers (or watchers) of the repo...	None
32	repo-packages-ecosystems	SGA-GL-0020	list	0	List of package platforms or services where th...	None
33	repo-packages-	SGA-GL-	int	0	Count of package	None

	name	id	result-type	sustainability_correlation	description	correction_guidance
	ecosystems-count	0021			platforms or services where t...	
34	repo-packages-versions-count	SGA-GL-0022	int	0	Count of package versions on package hosts or ...	None
35	repo-social-media-platforms	SGA-GL-0023	list	0	Social media platforms detected within the rea...	None
36	repo-social-media-platforms-count	SGA-GL-0024	int	0	Count of social media platforms detected withi...	None
37	repo-doi-valid-format	SGA-GL-0025	bool	1	Whether the DOI found in the CITATION.cff file...	DOI within the CITATION.cff file is not of a v...
38	repo-doi-https-resolvable	SGA-GL-0026	bool	1	Whether the DOI found in the CITATION.cff file...	DOI within the CITATION.cff file is not https ...
39	repo-doi-cited-by-count	SGA-GL-0027	int	0	How many other works cite the DOI for the repo...	None
40	repo-days-between-doi-publication-date-and-lat...	SGA-GL-0028	int	0	The number of days between the most recent com...	None
41	repo-gh-workflow-success-ratio	SGA-SF-0001	float	0	Ratio of succeeding workflow runs out of queri...	None
42	repo-gh-workflow-succeeding-runs	SGA-SF-0002	int	0	Number of succeeding workflow runs out of quer...	None

	name	id	result-type	sustainability_correlation	description	correction_guidance	
43	repo-gh-workflow-failing-runs	SGA-SF-0003	int	0	Number of failing workflow runs out of queried...	None	
44	repo-gh-workflow-queried-total	SGA-SF-0004	int	0	Total number of workflow runs from GitHub (onl...	None	
45	repo-code-coverage-percent	SGA-SF-0005	float	0	Percentage of code coverage for repository giv...	None	
46	repo-date-of-last-coverage-run	SGA-SF-0006	str	0	Date of code coverage run for repository given...	None	
47	repo-days-between-last-coverage-run-latest-commit	SGA-SF-0007	int	0	Days between last coverage run date and latest...	None	
48	repo-code-coverage-total-lines	SGA-SF-0008	int	0	Total lines of code used for code coverage wit...	None	
49	repo-code-coverage-executed-lines	SGA-SF-0009	int	0	Total lines covered code within repository giv...	None	
50	repo-agg-info-entropy	SGA-VS-0001	float	0	Aggregated information entropy for all files w...	None	
51	repo-file-info-entropy	SGA-VS-0002	dict	0	File-level information entropy for all files w...	None	{

```
# print the json with indentation
print(json.dumps(almanack_table, indent=4))
```

```
[
    {
        "name": "repo-path",
        "id": "SGA-META-0001",
        "result-type": "str",
        "sustainability_correlation": 0,
        "description": "Repository path (local directory).",
        "correction_guidance": null,
        "result": "/home/runner/work/almanac/almanac/src/_pdfbuild/seed-bank/almanack-example/a
    },
    {
        "name": "repo-commits",
        "id": "SGA-META-0002",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Total number of commits for the repository.",
        "correction_guidance": null,
        "result": 180
    },
    {
        "name": "repo-file-count",
        "id": "SGA-META-0003",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Total number of files tracked within the repository.",
        "correction_guidance": null,
        "result": 123
    },
    {
        "name": "repo-commit-time-range",
        "id": "SGA-META-0004",
        "result-type": "tuple",
        "sustainability_correlation": 0,
        "description": "Starting commit and most recent commit for the repository.",
        "correction_guidance": null,
        "result": [
            "2024-03-05",
            "2025-04-15"
        ]
    },
    {
        "name": "repo-days-of-development",
        "id": "SGA-META-0005",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Integer representing the number of days of development between most rec
        "correction_guidance": null,
        "result": 407
    },
    {
        "name": "repo-commits-per-day",
        "id": "SGA-META-0006",
        "result-type": "float",
        "sustainability_correlation": 0,
        "description": "Floating point number which represents the number of commits per day (u
        "correction_guidance": null,
        "result": 0.44226044226044225
    },
    {
        "name": "almanack-table-datetime",
```

```
        "id": "SGA-META-0007",
        "result-type": "str",
        "sustainability_correlation": 0,
        "description": "String representing the date when this table was generated in the forma
        "correction_guidance": null,
        "result": "2025-04-15T21:12:04.028623Z"
    },
    {
        "name": "almanack-version",
        "id": "SGA-META-0008",
        "result-type": "str",
        "sustainability_correlation": 0,
        "description": "String representing the version of the almanack which was used to gener
        "correction_guidance": null,
        "result": "0.0.0.post1.dev0+16e992f"
    },
    {
        "name": "repo-primary-language",
        "id": "SGA-META-0009",
        "result-type": "str",
        "sustainability_correlation": 0,
        "description": "Detected primary programming language of the repository.",
        "correction_guidance": null,
        "result": "Jupyter Notebook"
    },
    {
        "name": "repo-primary-license",
        "id": "SGA-META-0010",
        "result-type": "str",
        "sustainability_correlation": 0,
        "description": "Detected primary license of the repository.",
        "correction_guidance": null,
        "result": "bsd-3-clause"
    },
    {
        "name": "repo-doi",
        "id": "SGA-META-0011",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Repository DOI value detected from CITATION.cff file.",
        "correction_guidance": null,
        "result": "10.5281/zenodo.14765834"
    },
    {
        "name": "repo-doi-publication-date",
        "id": "SGA-META-0012",
        "result-type": "str",
        "sustainability_correlation": 0,
        "description": "Repository DOI publication date detected from CITATION.cff file and Ope
        "correction_guidance": null,
        "result": null
    },
    {
        "name": "repo-almanack-score",
        "id": "SGA-META-0013",
        "result-type": "dict",
        "sustainability_correlation": 0,
        "description": "Dictionary of length three, including the following: 1) number of Alman
        "correction_guidance": null,
        "result": {
            "almanack-score-numerator": 11,
```

```
            "almanack-score-denominator": 11,
            "almanack-score": 1.0
        }
    },
    {
        "name": "repo-includes-readme",
        "id": "SGA-GL-0001",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Boolean value indicating the presence of a README file in the repositor
        "correction_guidance": "Consider adding a README file to the repository.",
        "result": true
    },
    {
        "name": "repo-includes-contributing",
        "id": "SGA-GL-0002",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Boolean value indicating the presence of a CONTRIBUTING file in the rep
        "correction_guidance": "Consider adding a CONTRIBUTING file to the repository.",
        "result": true
    },
    {
        "name": "repo-includes-code-of-conduct",
        "id": "SGA-GL-0003",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Boolean value indicating the presence of a CODE_OF_CONDUCT file in the
        "correction_guidance": "Consider adding a CODE_OF_CONDUCT file to the repository.",
        "result": true
    },
    {
        "name": "repo-includes-license",
        "id": "SGA-GL-0004",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Boolean value indicating the presence of a LICENSE file in the reposito
        "correction_guidance": "Consider adding a LICENSE file to the repository.",
        "result": true
    },
    {
        "name": "repo-is-citable",
        "id": "SGA-GL-0005",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Boolean value indicating the presence of a CITATION file or some other
        "correction_guidance": "Consider adding a CITATION file to the repository (e.g. citatio
        "result": true
    },
    {
        "name": "repo-default-branch-not-master",
        "id": "SGA-GL-0006",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Boolean value indicating that the repo uses a default branch name besid
        "correction_guidance": "Consider using a default source control branch name other than
        "result": true
    },
    {
        "name": "repo-includes-common-docs",
        "id": "SGA-GL-0007",
```

```
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Boolean value indicating whether the repo includes common documentation
        "correction_guidance": "Consider including project documentation through the 'docs/' di
        "result": true
    },
    {
        "name": "repo-unique-contributors",
        "id": "SGA-GL-0008",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of unique contributors since the beginning of the repository.",
        "correction_guidance": null,
        "result": 5
    },
    {
        "name": "repo-unique-contributors-past-year",
        "id": "SGA-GL-0009",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of unique contributors within the last year from now (where now i
        "correction_guidance": null,
        "result": 4
    },
    {
        "name": "repo-unique-contributors-past-182-days",
        "id": "SGA-GL-0010",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of unique contributors within the last 182 days from now (where n
        "correction_guidance": null,
        "result": 2
    },
    {
        "name": "repo-tags-count",
        "id": "SGA-GL-0011",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of the number of tags within the repository.",
        "correction_guidance": null,
        "result": 7
    },
    {
        "name": "repo-tags-count-past-year",
        "id": "SGA-GL-0012",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of the number of tags within the repository within the last year
        "correction_guidance": null,
        "result": 7
    },
    {
        "name": "repo-tags-count-past-182-days",
        "id": "SGA-GL-0013",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of the number of tags within the repository within the last 182 d
        "correction_guidance": null,
        "result": 5
    },
    {
```

```
        "name": "repo-stargazers-count",
        "id": "SGA-GL-0014",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of the number of stargazers on repository remote hosting platform
        "correction_guidance": null,
        "result": 9
    },
    {
        "name": "repo-uses-issues",
        "id": "SGA-GL-0015",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Whether the repository uses issues (for example, as a bug and/or featur
        "correction_guidance": "Consider leveraging issues for tracking bugs and/or features wi
        "result": true
    },
    {
        "name": "repo-issues-open-count",
        "id": "SGA-GL-0016",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of open issues for repository.",
        "correction_guidance": null,
        "result": 43
    },
    {
        "name": "repo-pull-requests-enabled",
        "id": "SGA-GL-0017",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Whether the repository enables pull requests.",
        "correction_guidance": "Consider enabling pull requests for the repository through your
        "result": true
    },
    {
        "name": "repo-forks-count",
        "id": "SGA-GL-0018",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of forks of the repository.",
        "correction_guidance": null,
        "result": 2
    },
    {
        "name": "repo-subscribers-count",
        "id": "SGA-GL-0019",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of subscribers (or watchers) of the repository.",
        "correction_guidance": null,
        "result": 1
    },
    {
        "name": "repo-packages-ecosystems",
        "id": "SGA-GL-0020",
        "result-type": "list",
        "sustainability_correlation": 0,
        "description": "List of package platforms or services where the repository was detected
        "correction_guidance": null,
        "result": [
```

```
            "pypi"
        ]
    },
    {
        "name": "repo-packages-ecosystems-count",
        "id": "SGA-GL-0021",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of package platforms or services where the repository was detecte
        "correction_guidance": null,
        "result": 1
    },
    {
        "name": "repo-packages-versions-count",
        "id": "SGA-GL-0022",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of package versions on package hosts or services where the reposi
        "correction_guidance": null,
        "result": 8
    },
    {
        "name": "repo-social-media-platforms",
        "id": "SGA-GL-0023",
        "result-type": "list",
        "sustainability_correlation": 0,
        "description": "Social media platforms detected within the readme.",
        "correction_guidance": null,
        "result": []
    },
    {
        "name": "repo-social-media-platforms-count",
        "id": "SGA-GL-0024",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Count of social media platforms detected within the readme.",
        "correction_guidance": null,
        "result": 0
    },
    {
        "name": "repo-doi-valid-format",
        "id": "SGA-GL-0025",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Whether the DOI found in the CITATION.cff file is of a valid format.",
        "correction_guidance": "DOI within the CITATION.cff file is not of a valid format or mi
        "result": true
    },
    {
        "name": "repo-doi-https-resolvable",
        "id": "SGA-GL-0026",
        "result-type": "bool",
        "sustainability_correlation": 1,
        "description": "Whether the DOI found in the CITATION.cff file is https resolvable.",
        "correction_guidance": "DOI within the CITATION.cff file is not https resolvable or mis
        "result": true
    },
    {
        "name": "repo-doi-cited-by-count",
        "id": "SGA-GL-0027",
        "result-type": "int",
```

```
        "sustainability_correlation": 0,
        "description": "How many other works cite the DOI for the repository found within the C
        "correction_guidance": null,
        "result": null
    },
    {
        "name": "repo-days-between-doi-publication-date-and-latest-commit",
        "id": "SGA-GL-0028",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "The number of days between the most recent commit and DOI for the repos
        "correction_guidance": null,
        "result": null
    },
    {
        "name": "repo-gh-workflow-success-ratio",
        "id": "SGA-SF-0001",
        "result-type": "float",
        "sustainability_correlation": 0,
        "description": "Ratio of succeeding workflow runs out of queried workflow runs from Git
        "correction_guidance": null,
        "result": 0.83
    },
    {
        "name": "repo-gh-workflow-succeeding-runs",
        "id": "SGA-SF-0002",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Number of succeeding workflow runs out of queried workflow runs from Gi
        "correction_guidance": null,
        "result": 100
    },
    {
        "name": "repo-gh-workflow-failing-runs",
        "id": "SGA-SF-0003",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Number of failing workflow runs out of queried workflow runs from GitHu
        "correction_guidance": null,
        "result": 83
    },
    {
        "name": "repo-gh-workflow-queried-total",
        "id": "SGA-SF-0004",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Total number of workflow runs from GitHub (only applies to GitHub hoste
        "correction_guidance": null,
        "result": 17
    },
    {
        "name": "repo-code-coverage-percent",
        "id": "SGA-SF-0005",
        "result-type": "float",
        "sustainability_correlation": 0,
        "description": "Percentage of code coverage for repository given detected code coverage
        "correction_guidance": null,
        "result": null
    },
    {
        "name": "repo-date-of-last-coverage-run",
```

```
        "id": "SGA-SF-0006",
        "result-type": "str",
        "sustainability_correlation": 0,
        "description": "Date of code coverage run for repository given detected code coverage d
        "correction_guidance": null,
        "result": null
    },
    {
        "name": "repo-days-between-last-coverage-run-latest-commit",
        "id": "SGA-SF-0007",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Days between last coverage run date and latest commit date.",
        "correction_guidance": null,
        "result": null
    },
    {
        "name": "repo-code-coverage-total-lines",
        "id": "SGA-SF-0008",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Total lines of code used for code coverage within repository given dete
        "correction_guidance": null,
        "result": null
    },
    {
        "name": "repo-code-coverage-executed-lines",
        "id": "SGA-SF-0009",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Total lines covered code within repository given detected code coverage
        "correction_guidance": null,
        "result": null
    },
    {
        "name": "repo-agg-info-entropy",
        "id": "SGA-VS-0001",
        "result-type": "float",
        "sustainability_correlation": 0,
        "description": "Aggregated information entropy for all files within a repository given
        "correction_guidance": null,
        "result": 0.0034933186678297494
    },
    {
        "name": "repo-file-info-entropy",
        "id": "SGA-VS-0002",
        "result-type": "dict",
        "sustainability_correlation": 0,
        "description": "File-level information entropy for all files within a repository given
        "correction_guidance": null,
        "result": {
            ".alexignore": 8.09597997105849e-05,
            ".github/ISSUE_TEMPLATE/bug.yml": 0.0022919940085864045,
            ".github/ISSUE_TEMPLATE/config.yml": 8.09597997105849e-05,
            ".github/ISSUE_TEMPLATE/feature.yml": 0.0020652431030483993,
            ".github/PULL_REQUEST_TEMPLATE.md": 0.0011699291446290996,
            ".github/actions/install-node-env/action.yml": 0.0004770429144282104 6,
            ".github/actions/install-python-env/action.yml": 0.0009469720953823388,
            ".github/dependabot.yml": 0.0011147610106358615,
            ".github/release-drafter.yml": 0.0006875360779865752,
            ".github/workflows/deploy-book.yml": 0.0011973820889953737,
```

```
".github/workflows/draft-release.yml": 0.0007461282554497849,
".github/workflows/entropy-check.yml": 0.0018347792995470958,
".github/workflows/pre-commit-checks.yml": 0.0010870410067802619,
".github/workflows/publish-pypi.yml": 0.0008616389747532617,
".github/workflows/pytest-tests.yml": 0.0011423895658537245,
".gitignore": 0.003871157868674245,
".linkcheckerrc.ini": 0.0005684015482974668,
".pre-commit-config.yaml": 0.004798626950586306,
".pre-commit-hooks.yaml": 0.00041498299105245467,
".vale.ini": 0.00047704291442821046,
"CITATION.cff": 0.0048208510954034,
"CODE_OF_CONDUCT.md": 0.0001174356576150573,
"CONTRIBUTING.md": 0.0001174356576150573,
"LICENSE": 0.000890199582880684,
"LICENSE.txt": 0.000890199582880684,
"README.md": 0.0026871764038194565,
"coverage.xml": 0.021877102117648997,
"docs/readme.md": 0.0001174356576150573,
"media/coverage-badge.svg": 4.2761551497283055e-05,
"pa11y.json": 0.00025449301870803946,
"package-lock.json": 0.028815043491479557,
"package.json": 0.00018731860223499596,
"poetry.lock": 0.06388401503682185,
"pyproject.toml": 0.004731849084966107,
"src/almanack/__init__.py": 0.0007752063426971347,
"src/almanack/book.py": 0.0018605850653017236,
"src/almanack/cli.py": 0.004798626950586306,
"src/almanack/git.py": 0.008997987521546789,
"src/almanack/metrics/data.py": 0.01735340118536627,
"src/almanack/metrics/entropy/calculate_entropy.py": 0.002953719144343076,
"src/almanack/metrics/entropy/processing_repositories.py": 0.0019633015698791454,
"src/almanack/metrics/garden_lattice/connectedness.py": 0.00665399774373518,
"src/almanack/metrics/garden_lattice/practicality.py": 0.0030975255525692072,
"src/almanack/metrics/garden_lattice/understanding.py": 0.001652626673809894,
"src/almanack/metrics/metrics.yml": 0.009979803419108597,
"src/almanack/metrics/remote.py": 0.0024907703476468365,
"src/almanack/reporting/report.py": 0.0032402919874516102,
"src/book/_config.yml": 0.0014141319919936932,
"src/book/_static/OSSci Monthly Call Jan 2025 - Software_Gardening_Almanack.odp": 0
"src/book/_static/OSSci Monthly Call Jan 2025 - Software_Gardening_Almanack.pdf": 0
"src/book/_static/custom.css": 0.0004461396827978894,
"src/book/_toc.yml": 0.0010592269407672533,
"src/book/assets/640px-Forgard2-003.gif": 0.0,
"src/book/assets/640px-Rundes_Fenster_mit_Gitter.jpeg": 0.0,
"src/book/assets/Sundial_2916_HDR.jpeg": 0.0,
"src/book/assets/almanack-influencing-software.png": 0.0,
"src/book/assets/garden-lattice-understanding-transfer.png": 0.0,
"src/book/assets/software-gardening-almanack-logo.png": 0.0,
"src/book/assets/software-gardening-logo.png": 0.0,
"src/book/assets/software-lifecycle.png": 0.0,
"src/book/assets/xkcd_dependency.png": 0.0,
"src/book/favicon.png": 0.0,
"src/book/garden-circle/contributing.md": 0.004687241699561337,
"src/book/garden-circle/garden-circle.md": 0.00018731860223499596,
"src/book/garden-circle/garden-map.md": 0.0013334213367324893,
"src/book/garden-circle/package-api.md": 0.003025755813119874,
"src/book/garden-circle/pavilion.md": 0.00038355167030691565,
"src/book/garden-lattice/garden-lattice.md": 0.0012792427870095449,
"src/book/garden-lattice/understanding.md": 0.003169035388791238,
"src/book/introduction.md": 0.003940185819369114,
"src/book/references.bib": 0.007257910512931364,
```

```
            "src/book/seed-bank/almanack-example/almanack-example.ipynb": 0.03360489040759997,
            "src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/generate_data.py
            "src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/generate_github_
            "src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/pubmed_github_li
            "src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/pubmed_github_li
            "src/book/seed-bank/pubmed-github-repositories/gather-software-information-entropy.
            "src/book/seed-bank/pubmed-github-repositories/images/pubmed-lines-of-code-and-time
            "src/book/seed-bank/pubmed-github-repositories/images/pubmed-stars-and-forks.png":
            "src/book/seed-bank/pubmed-github-repositories/images/pubmed-stars-and-open-issues.
            "src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-
            "src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-
            "src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-
            "src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/reposito
            "src/book/seed-bank/pubmed-github-repositories/visualize-pubmed-repo-sofware-entrop
            "src/book/seed-bank/seed-bank.md": 0.00018731860223499596,
            "src/book/software-forest/software-forest.md": 0.0013063703412231746,
            "src/book/verdant-sundial/verdant-sundial.md": 0.0013334213367324893,
            "styles/config/vocabularies/almanack/accept.txt": 0.0008041472349178831,
            "tests/__init__.py": 0.0,
            "tests/conftest.py": 0.004485606421280154,
            "tests/data/almanack/coverage/python/coverage.json": 4.2761551497283055e-05,
            "tests/data/almanack/coverage/python/coverage.lcov": 0.020006370398135475,
            "tests/data/almanack/repo_setup/create_repo.py": 0.006569929420618511,
            "tests/data/almanack/repo_setup/insert_code.py": 0.001652626673809894,
            "tests/data/jupyter-book/sandbox.md": 0.0005381591188691302,
            "tests/metrics/test_calculate_entropy.py": 0.0016788169198780679,
            "tests/metrics/test_data.py": 0.022155169267006697,
            "tests/metrics/test_garden_lattice.py": 0.002266969110322942,
            "tests/test_almanack.py": 0.0010870410067802619,
            "tests/test_build.py": 0.0018863393957209958,
            "tests/test_cli.py": 0.003025755813119874,
            "tests/test_git.py": 0.009195961458088785,
            "tests/utils.py": 0.0010870410067802619
        }
    }
]
```

```
%%time

# show the same results from the CLI
!almanack table "./almanack"
```


```
[{"name": "repo-path", "id": "SGA-META-0001", "result-type": "str", "sustainability_correlation
```


```
CPU times: user 46.1 ms, sys: 14.4 ms, total: 60.5 ms
Wall time: 4.18 s
```

```
%%time

# show the same results from the CLI
!almanack check "./almanack"
```

```
Running Software Gardening Almanack checks.
Datetime: 2025-04-15T21:12:15.534197Z
Almanack version: 0.0.0.post1.dev0+16e992f
Target repository path: ./almanack
```

```
Software Gardening Almanack score: 100.00% (11/11)
```

```
CPU times: user 40.7 ms, sys: 17.3 ms, total: 58 ms
Wall time: 3.85 s
```

Visualizing PubMed article GitHub repository software information entropy

The content below seeks to better understand how software information entropy manifests in a dataset of ~10,000 PubMed article GitHub repositories.

PubMed GitHub repositories are extracted using the PubMed API to query for GitHub links within article abstracts. GitHub data about these repositories is gathered using the GitHub API. The code to perform data extractions may be found under the directory: gather-pubmed-repos .

Software entropy measurements are gathered using the notebook: software-information-entropy.ipynb .

We derive software information entropy using methods inspired from *Predicting faults using the complexity of code changes*[SB1]. Software information entropy within the context of this notebook is normalized for all files within a repository using the first and latest commits.

Project durations highlighted below is the number of days between the first and latest commit date.

Data Extraction

The following section extracts data which includes software entropy and GitHub-derived data. We merge the data to form a table which includes PubMed, GitHub, and Almanack software entropy information on the repositories.

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.io as pio

# set plotly default theme
pio.templates.default = "plotly_white"

# read example data which includes pubmed github links detected from article abstracts
df = pd.merge(
    left=pd.read_parquet("repository_analysis_results"),
    right=pd.read_parquet(
        "gather-pubmed-repos/pubmed_github_links_with_github_data.parquet"
    ),
    left_on="Repository URL",
    right_on="github_link",
)
df
```

	Repository URL	Normalized Total Entropy	Date of First Commit	Date of Last Commit	Time of Existence (days)	F
0	https://github.com/BUStools/BUSZ-format	0.000000	2022-11-08	2022-11-18	10.0	3712
1	https://github.com/pmelsted/BUSZ_paper	0.178897	2023-03-27	2023-03-28	1.0	3712
2	https://github.com/WormBase/scdefg	0.153966	2021-01-29	2022-02-02	368.0	3581
3	https://github.com/ekg/guix-genomics	0.176469	2020-01-06	2024-01-22	1476.0	3644
4	https://github.com/ekg/seqwish	0.027122	2018-06-11	2023-12-09	2007.0	3644
...	...	...	...	...	...	
11153	https://github.com/maxplanck-ie/parkour	0.000449	2016-05-23	2022-08-24	2284.0	3023
11154	https://github.com/linsalrob/partie	0.000257	2016-09-19	2022-10-23	2224.0	2836

	Repository URL	Normalized Total Entropy	Date of First Commit	Date of Last Commit	Time of Existence (days)	F
11155	https://github.com/louzounlab/CountingIsAlmost...	0.011558	2022-06-15	2022-11-04	141.0	3674
11156	https://github.com/sysbio-polito/NWN_CElegans_...	0.002924	2021-02-12	2021-10-06	235.0	3476
11157	https://github.com/lennylv/DGCddG	0.014386	2021-09-26	2024-06-12	990.0	3701

11158 rows × 33 columns

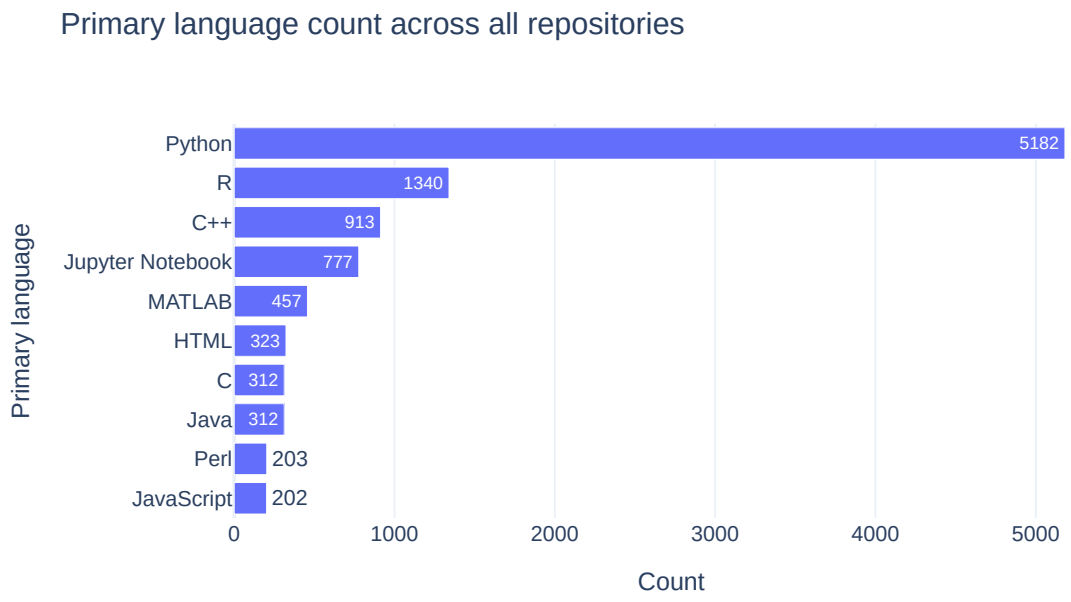
What languages are used within PubMed article repositories?

The following section observes the top 10 languages which are used in repositories from the dataset. Primary language is determined as the language which has the most lines of code within a repository.

```
language_grouped_data = (
    df.groupby(["Primary language"]).size().reset_index(name="Count")
)

# Create a horizontal bar chart
fig_languages = px.bar(
    language_grouped_data.sort_values(by="Count")[-10:],
    y="Primary language",
    x="Count",
    text="Count",
    orientation="h",
    width=700,
    height=400,
    title="Primary language count across all repositories",
)

fig_languages.show()
```

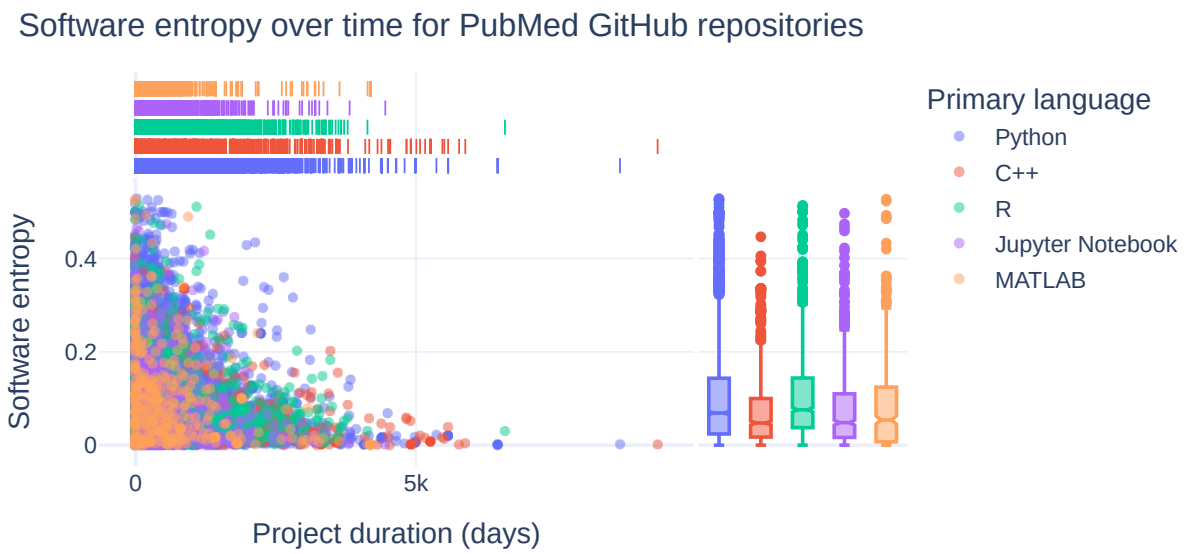


How is software entropy different across primary languages?

The following section explores how software entropy manifests differently across different primary languages for repositories.

```
fig = px.scatter(
    df[
        df["Primary language"].isin(
            language_grouped_data.sort_values(by="Count")[-5:]["Primary language"]
        )
    ],
    x="Time of Existence (days)",
    y="Normalized Total Entropy",
    hover_data=["Repository URL"],
    width=700,
    height=400,
    title="Software entropy over time for PubMed GitHub repositories",
    marginal_x="rug",
    marginal_y="box",
    opacity=0.5,
    color="Primary language",
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.8, "yanchor": "bottom"},
    xaxis_title="Project duration (days)",
    yaxis_title="Software entropy",
)
fig.write_image("images/software-information-entropy-top-5-langs.png")
fig.show()
```

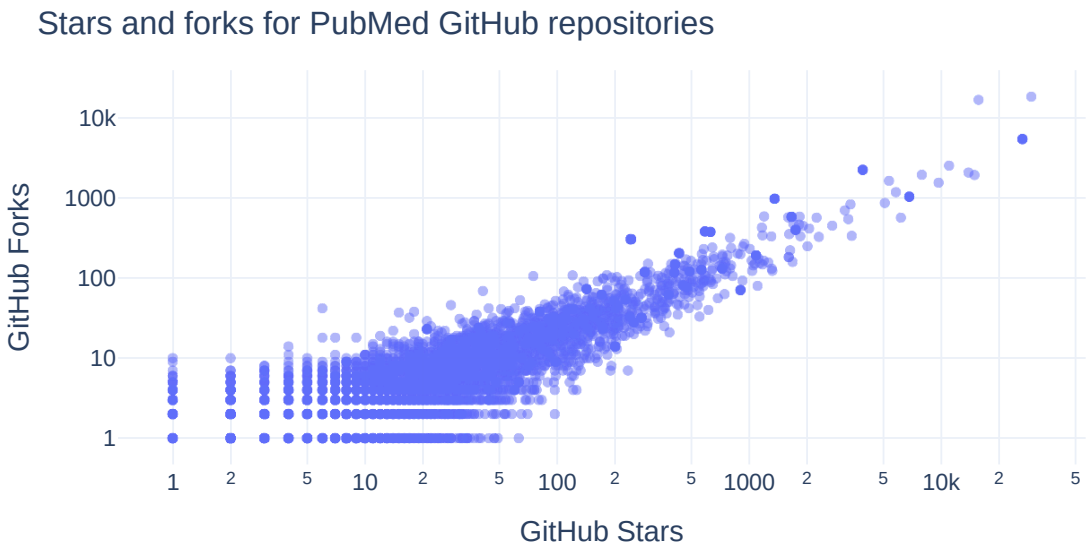


What is the relationship between GitHub Stars and Forks for repositories?

We next explore how GitHub Stars and Forks are related within the repositories.

```
fig = px.scatter(
    df,
    x="GitHub Stars",
    y="GitHub Forks",
    hover_data=["Repository URL"],
    width=700,
    height=400,
    title="Stars and forks for PubMed GitHub repositories",
    opacity=0.5,
    log_y=True,
    log_x=True,
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.8, "yanchor": "bottom"},
    xaxis_title="GitHub Stars",
    yaxis_title="GitHub Forks",
)
fig.write_image("images/pubmed-stars-and-forks.png")
fig.show()
```



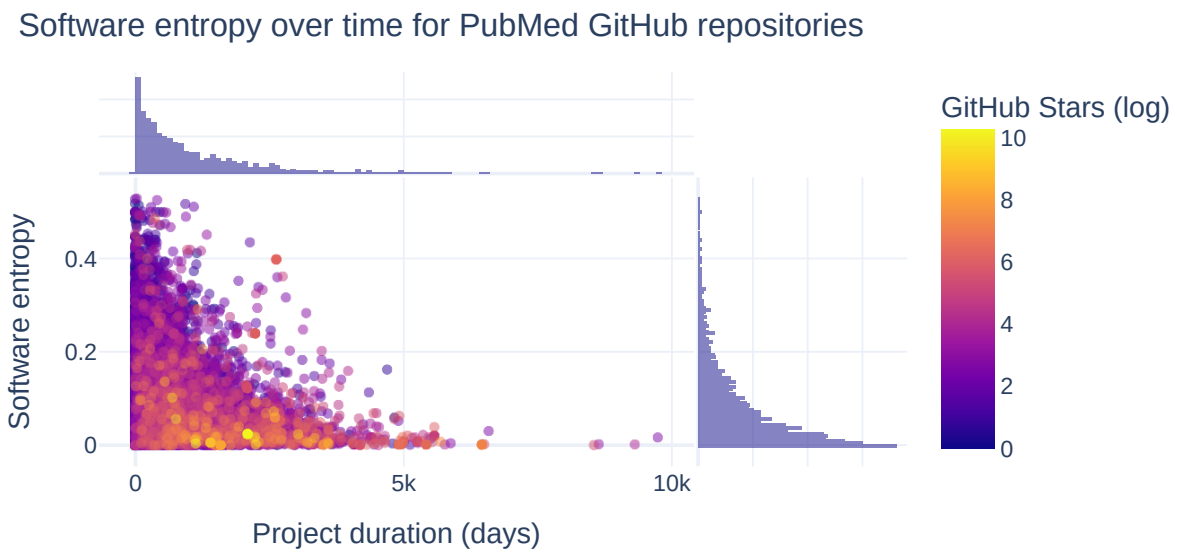
How do GitHub Stars, software entropy, and time relate?

The next section explores how GitHub Stars, software entropy, and time relate to one another.

```
df["GitHub Stars (log)"] = np.log(
    df["GitHub Stars"].apply(
        # move 0's to None to avoid divide by 0
        lambda x: x if x > 0 else None
    )
)

fig = px.scatter(
    df.dropna(subset="GitHub Stars (log)").sort_values(by="GitHub Stars (log)"),
    x="Time of Existence (days)",
    y="Normalized Total Entropy",
    hover_data=["Repository URL"],
    width=700,
    height=400,
    title="Software entropy over time for PubMed GitHub repositories",
    marginal_x="histogram",
    marginal_y="histogram",
    opacity=0.5,
    color="GitHub Stars (log)",
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.8, "yanchor": "bottom"},
    xaxis_title="Project duration (days)",
    yaxis_title="Software entropy",
)
fig.write_image("images/software-information-entropy-gh-stars.png")
fig.show()
```



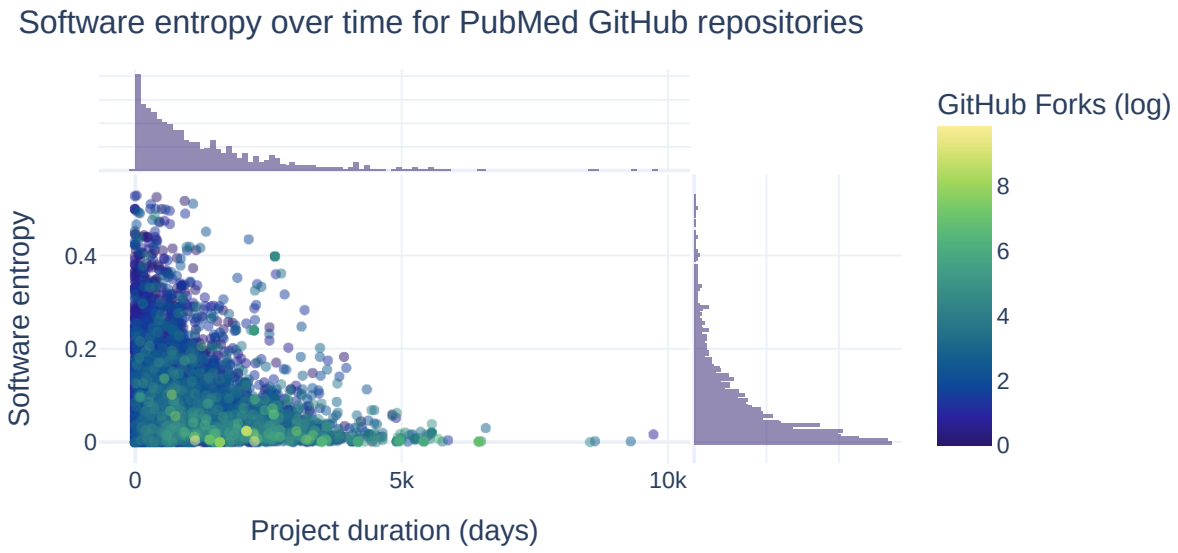
How do GitHub Forks, software entropy, and time relate?

The next section explores how GitHub Forks, software entropy, and time relate to one another.

```
df["GitHub Forks (log)"] = np.log(
    df["GitHub Forks"].apply(
        # move 0's to None to avoid divide by 0
        lambda x: x if x > 0 else None
    )
)

fig = px.scatter(
    df.dropna(subset="GitHub Forks (log)").sort_values(by="GitHub Forks (log)"),
    x="Time of Existence (days)",
    y="Normalized Total Entropy",
    hover_data=["Repository URL"],
    width=700,
    height=400,
    title="Software entropy over time for PubMed GitHub repositories",
    marginal_x="histogram",
    marginal_y="histogram",
    opacity=0.5,
    color="GitHub Forks (log)",
    color_continuous_scale=px.colors.sequential.haline,
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.8, "yanchor": "bottom"},
    xaxis_title="Project duration (days)",
    yaxis_title="Software entropy",
)
fig.write_image("images/software-information-entropy-forks.png")
fig.show()
```

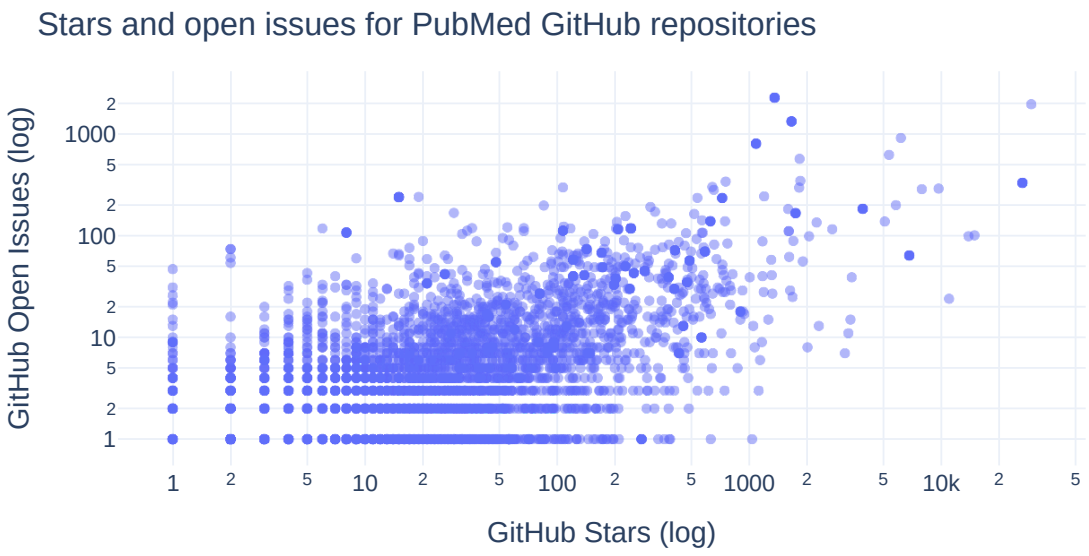


What is the relationship between GitHub Stars and Open Issues for the repositories?

Below we explore how GitHub Stars and Open Issues are related for the repositories.

```
df["GitHub Open Issues (log)"] = np.log(
    df["GitHub Open Issues"].apply(
        # move 0's to None to avoid divide by 0
        lambda x: x if x > 0 else None
    )
)
fig = px.scatter(
    df,
    x="GitHub Stars",
    y="GitHub Open Issues",
    hover_data=["Repository URL"],
    width=700,
    height=400,
    title="Stars and open issues for PubMed GitHub repositories",
    opacity=0.5,
    log_y=True,
    log_x=True,
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.8, "yanchor": "bottom"},
    xaxis_title="GitHub Stars (log)",
    yaxis_title="GitHub Open Issues (log)",
)
fig.write_image("images/pubmed-stars-and-open-issues.png")
fig.show()
```



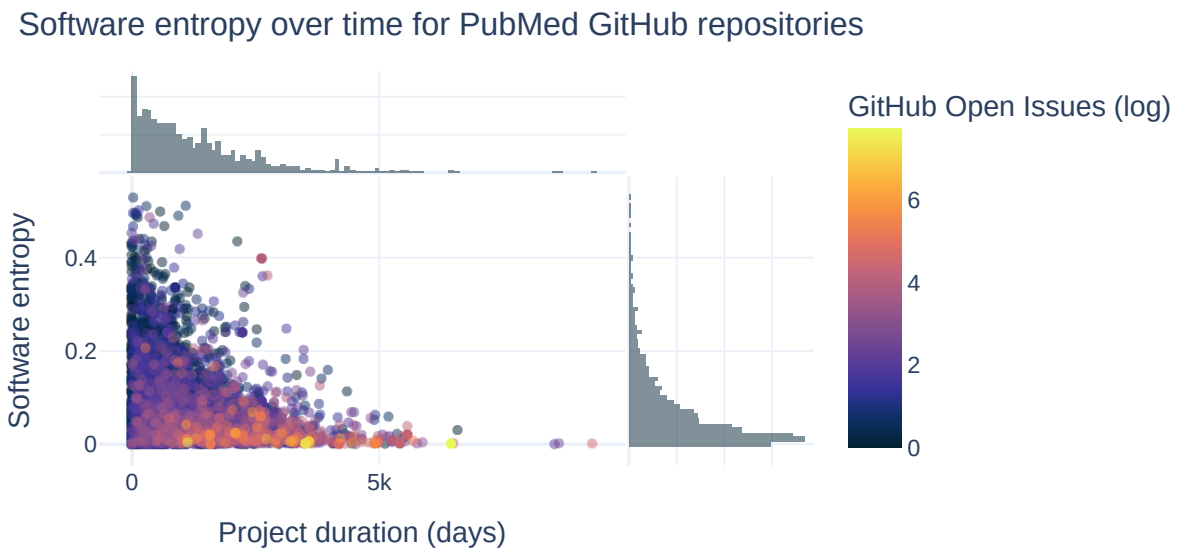
How do software entropy, time, and GitHub issues relate to one another?

The next section visualizes how software entropy, time, and GitHub issues relate to one another.

```
df["GitHub Open Issues (log)"] = np.log(
    df["GitHub Open Issues"].apply(
        # move 0's to None to avoid divide by 0
        lambda x: x if x > 0 else None
    )
)

fig = px.scatter(
    df.dropna(subset="GitHub Open Issues (log)").sort_values(
        by="GitHub Open Issues (log)"
    ),
    x="Time of Existence (days)",
    y="Normalized Total Entropy",
    hover_data=["Repository URL"],
    width=700,
    height=400,
    title="Software entropy over time for PubMed GitHub repositories",
    marginal_x="histogram",
    marginal_y="histogram",
    opacity=0.5,
    color="GitHub Open Issues (log)",
    color_continuous_scale=px.colors.sequential.thermal,
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.8, "yanchor": "bottom"},
    xaxis_title="Project duration (days)",
    yaxis_title="Software entropy",
)
fig.write_image("images/software-information-entropy-open-issues.png")
fig.show()
```



[SB1] Ahmed E. Hassan. Predicting faults using the complexity of code changes. In *2009 IEEE 31st International Conference on Software Engineering*, 78–88. Vancouver, BC, Canada, 2009. IEEE. URL: http://ieeexplore.ieee.org/document/5070510 (visited on 2024-07-15), doi:10.1109/ICSE.2009.5070510.

Garden Circle

The garden circle is a section of the Software Garden Almanack associated with contributions, planning, and internal maintenance of the project.

Contributing

First of all, thank you for contributing to the Software Gardening Almanack! 🎉 💯 🪴 We're so grateful for the kindness of your effort applied to grow this project into the best that it can be.

This document contains guidelines on how to most effectively contribute to this project.

If you would like clarifications, please feel free to ask any questions or for help. We suggest asking for help from GitHub where you may need it (for example, in a GitHub issue or a pull request) by "at (@) mentioning" a Software Gardening Almanack maintainer (for example, `@username`).

Code of conduct

Our Code of Conduct (CoC) policy (located here) governs this project. By participating in this project, we expect you to uphold this code. Please report unacceptable behavior by following the procedures listed under the CoC Enforcement section.

Security

Please see our Security policy (located here) for more information on security practices and recommendations associated with this project.

Quick links

- Documentation: software-gardening/almanack
- Issue tracker: software-gardening/almanack#issues

Process

Reporting bugs or suggesting enhancements

We're deeply committed to a smooth and intuitive user experience which helps people benefit from the content found within this project. This commitment requires a good relationship and open communication with our users.

We encourage you to file a GitHub issue to report bugs or propose enhancements to improve the Software Gardening Almanack.

First, figure out if your idea is already implemented by reading existing issues or pull requests! Check the issues (software-gardening/almanack#issues) and pull requests (software-gardening/almanack) to see if someone else has already documented or began implementation of your idea. If you do find your idea in an existing issue, please comment on the existing issue noting that you are also interested in the functionality. If you do not find your idea, please open a new issue and document why it would be helpful for your particular use case.

Please also provide the following specifics:

- The version of the Software Gardening Almanack you're referencing.
- Specific error messages or how the issue exhibits itself.
- Operating system (e.g. MacOS, Windows, etc.)
- Device type (e.g. laptop, phone, etc.)
- Any examples of similar capabilities

Your first code contribution

Contributing code for the first time can be a daunting task. However, in our community, we strive to be as welcoming as possible to newcomers, while ensuring sustainable software development practices.

The first thing to figure out is specifically what you're going to contribute! We describe all future work as individual GitHub issues. For first time contributors we have specifically labeled as good first issue.

If you want to contribute code that we haven't already outlined, please start a discussion in a new issue before writing any code. A discussion will clarify the new code and reduce merge time. Plus, it's possible your contribution belongs in a different code base, and we do not want to waste your time (or ours)!

Pull requests

After you've decided to contribute code and have written it up, please file a pull request. We specifically follow a forked pull request model. Please create a fork of Software Gardening Almanack repository, clone the fork, and then create a new, feature-specific branch. Once you make the necessary changes on this branch, you should file a pull request to incorporate your changes into the main The Software Gardening Almanack repository.

The content and description of your pull request are directly related to the speed at which we are able to review, approve, and merge your contribution into The Software Gardening Almanack. To ensure an efficient review process please perform the following steps:

1. Follow all instructions in the pull request template
2. Triple check that your pull request is adding *one* specific feature. Small, bite-sized pull requests move so much faster than large pull requests.
3. After submitting your pull request, ensure that your contribution passes all status checks (e.g. passes all tests)

We require pull request review and approval by at least one project maintainer in order to merge. We will do our best to review the code additions in as soon as we can. Ensuring that you follow all steps above will increase our speed and ability to review. We will check for accuracy, style, code coverage, and scope.

Git commit messages

For all commit messages, please use a short phrase that describes the specific change. For example, "Add feature to check normalization method string" is much preferred to "change code". When appropriate, reference issues (via `#` plus number) .

Development

Overview

We write and develop the Software Gardening Almanack in Python through Jupyter Book with related environments managed by Python Poetry. We use Node and NPM dependencies to assist development activities (such as performing linting checks or other capabilities). We use GitHub actions for CI/CD procedures such as automated tests.

Getting started

To enable local development, perform the following steps.

1. Install Python (suggested: use `pyenv` for managing Python versions)
2. Install Poetry
3. Install Poetry environment: `poetry install`
4. Install Node (suggested: use `nvm` for managing Node versions)
5. Install Node environment: `npm install`
6. Install Vale dependencies: `poetry run vale sync`
7. Optional: install book PDF rendering dependencies: `poetry run playwright install --with-deps chromium`

Development Tasks

We use Poe the Poet to define common development tasks, which simplifies repeated commands. We include Poe the Poet as a Python Poetry `dev` group dependency, which users access through the Poetry environment. Please see the `pyproject.toml` file's `[tool.poe.tasks]` table for a list of available tasks.

For example:

```
# example of how Poe the Poet commands may be used.
poetry run poe <task_name>

# example of a task which runs jupyter-book build for the project
poetry run poe build-book
```

Linting

pre-commit automatically checks all work added to this repository. We implement these checks using GitHub Actions. Pre-commit can work alongside your local git with git-hooks After installing pre-commit within your

development environment, the following command performs the same checks:

```
% pre-commit run --all-files
```

We strive to provide comments about each pre-commit check within the `.pre-commit-config.yaml` file. Comments are provided within the file to make sure we single-source documentation where possible, avoiding possible drift between the documentation and the code which runs the checks.

Automated CI/CD with GitHub Actions

We use GitHub Actions to help perform automated CI/CD as part of this project. GitHub Actions involves defining workflows through YAML files. These workflows include one or more jobs which are collections of individual processes (or steps) which run as part of a job. We define GitHub Actions work under the `.github` directory. We suggest the use of `act` to help test GitHub Actions work during development.

Releases

We utilize semantic versioning ("semver") to distinguish between major, minor, and patch releases. We publish source code by using GitHub Releases available here. Contents of the book are distributed as both a website and PDF. We distribute a Python package through the Python Packaging Index (PyPI) available here which both includes and provides tooling for applying the book's content.

Publishing Software Gardening Almanack releases involves several manual and automated steps. See below for an overview of how this works.

Version specifications

We follow semantic version (semver) specifications with this project through the following technologies.

- `poetry-dynamic-versioning` leveraging `dunamai` creates version data based on git tags and commits.
- GitHub Releases automatically create git tags and source code collections available from the GitHub repository.
- `release-drafter` infers and describes changes since last release within automatically drafted GitHub Releases after pull requests are merged (draft releases are published as decided by maintainers).

Release process

1. Open a pull request and use a repository label for `release-<semver release type>` to label the pull request for visibility with `release-drafter` (for example, see almanack#43 as a reference of a semver patch update).

2. On merging the pull request for the release, a GitHub Actions workflow defined in `draft-release.yml` leveraging `release-drafter` will draft a release for maintainers.
3. The draft GitHub release will include a version tag based on the GitHub PR label applied and `release-drafter`.
4. Make modifications as necessary to the draft GitHub release, then publish the release (the draft release does not normally need additional modifications).
5. On publishing the release, another GitHub Actions workflow defined in `publish-pypi.yml` will automatically build and deploy the Python package to PyPI (utilizing the earlier modified `pyproject.toml` semantic version reference for labeling the release).
6. Each GitHub release will trigger a Zenodo GitHub integration which creates a new Zenodo record and unique DOI.

Citations

We create a unique DOI per release through the Zenodo GitHub integration. As part of this we also use a special DOI provided from Zenodo to reference the latest record on each new release. Zenodo outlines how this works within their versioning documentation.

The DOI and other citation metadata are stored within a `CITATION.cff` file. Our `CITATION.cff` is the primary source of citation information in correspondence with this project. The `CITATION.cff` file is used within a sidebar integration through the GitHub web interface to enable people to directly cite this project.

Attribution

We sourced portions of this contribution guide from `pyctyominer`. Big thanks go to the developers and contributors of that repository.

Garden Map

Our garden map is where you can learn about what features we're working on, what stage they're in, and when we expect to bring them to you. The content here follows that of a technical roadmap (providing strategic visibility on implementation details and timelines).

Ethos

We intend for the garden map to provide useful tools for us to efficiently create and maintain the Software Gardening Almanack. We practice keeping the garden map as "petal-light" as possible through built-in tools, automation, and an avoidance of analysis paralysis where possible.

We'd love your input

We welcome your input on the garden map and elements found within it! Please see the contributing guide for more information on how to participate in this process.

Active garden map(s)

The following are active garden maps for the Software Gardening Almanack. Please see below for a further description of these projects.

- **2024 BSSw Fellowship Milestones**: this project communicates activities associated with the 2024 BSSw Fellowship associated with the Software Gardening Almanack.

Sections

You can explore the garden map with different views:

- **"Canopy-view" development visibility**: We track new, existing, and previous work at a high-level using organization-wide GitHub Projects.
- **"Ground-level" development visibility**: We track repository-specific work using GitHub issues and a GitHub Project field called "status", which we associate with each issue (indicated as "Todo", "In progress", "Paused", or "Done").
- **Major milestones**: We track major milestones using common GitHub issues labels across all repositories through their associated issues.

Visualizations

It can be helpful to visualize the garden map using data plots or other images. We use GitHub Projects insights to assist with automated creation and updates to garden map activity. Please see this project insights page for an example of these (using the insights button within any GitHub Projects for viewing others).

Acknowledgments

We drew inspiration for this content from the GitHub public roadmap. Big thanks to the original works found there.

Pavilion

We sometimes present the Software Gardening Almanack at events, which we want to share with you under this pavilion.

OSSci Monthly Call Jan 2025

The OpenSource.science (OSSci) Map of Open Source (MOSS) interest group invited us to present on the Almanack on 1/13/2025.

Please see the presentation recording or slides below as a reference to this presentation.

`almanack` Package API

Base

Module for interacting with Software Gardening Almanack book content through a Python package.

`almanack.book.read`(*`chapter_name: str | None = None`*)

A function for reading almanack content through a package interface.

Parameters:

chapter_name – Optional[str], default None A string which indicates the short-hand name of a chapter from the book. Short-hand names are lower-case title names read from *src/book/_toc.yml*.

Returns:

None

The outcome of this function involves printing the content of the selected chapter from the short-hand name or showing available chapter names through an exception.

Setup almanack CLI through python-fire

class almanack.cli.AlmanackCLI

Bases: `object`

Almanack CLI class for Google Fire

The following CLI-based commands are available (and in alignment with the methods below based on their name):

- ***almanack table <repo path>*: Provides a JSON data**

  structure which includes Almanack metric data. Always returns a 0 exit.

- ***almanack check <repo path>*: Provides a report**

  of boolean metrics which include a non-zero sustainability direction ("checks") that are failing to inform a user whether they pass. Returns non-zero exit (1) if any checks are failing, otherwise 0.

check(*repo_path: str, ignore: List[str] | None = None*) → None

Used through CLI to check table of metrics for boolean values with a non-zero sustainability direction for failures.

This enables the use of CLI such as: *almanack check <repo path>*

Parameters:

- **repo_path** (*str*) – The path to the repository to analyze.
- **ignore** (*List[str]*) – A list of metric IDs to ignore when running the checks. Defaults to None.

table(*repo_path: str, ignore: List[str] | None = None*) → None

Used through CLI to generate a table of metrics

This enables the use of CLI such as: *almanack table <repo path>*

Parameters:

- **repo_path** (*str*) – The path to the repository to analyze.
- **ignore** (*List[str]*) – A list of metric IDs to ignore when running the checks. Defaults to None.

`almanack.cli.trigger()`

Trigger the CLI to run.

This module performs git operations

`almanack.git.clone_repository(repo_url: str) → Path`

Clones the GitHub repository to a temporary directory.

Parameters:

repo_url (*str*) – The URL of the GitHub repository.

Returns:

Path to the cloned repository.

Return type:

pathlib.Path

`almanack.git.count_files(tree: Tree | Blob) → int`

Counts all files (Blobs) within a Git tree, including files in subdirectories.

This function recursively traverses the provided *tree* object to count each file, represented as a *pygit2.Blob*, within the tree and any nested subdirectories.

Parameters:

tree (*Union[pygit2.Tree, pygit2.Blob]*) – The Git tree object (of type *pygit2.Tree*) to traverse and count files. The initial call should be made with the root tree of a commit.

Returns:

The total count of files (Blobs) within the tree, including nested files in subdirectories.

Return type:

int

`almanack.git.detect_encoding(blob_data: bytes) → str`

Detect the encoding of the given blob data using charset-normalizer.

Parameters:

blob_data (*bytes*) – The raw bytes of the blob to analyze.

Returns:

The best detected encoding of the blob data.

Return type:

str

Raises:

ValueError – If no encoding could be detected.

almanack.git.file_exists_in_repo(*repo: Repository*, *expected_file_name: str*, *check_extension: bool = False*, *extensions: list[str] = ['.md', '.txt', '.rtf', '']*) → bool

Check if a file (case-insensitive and with optional extensions) exists in the latest commit of the repository.

Parameters:

- **repo** (*pygit2.Repository*) – The repository object to search in.
- **expected_file_name** (*str*) – The base file name to check (e.g., “readme”).
- **check_extension** (*bool*) – Whether to check the extension of the file or not.
- **extensions** (*list[str]*) – List of possible file extensions to check (e.g., [“.md”, “”]).

Returns:

True if the file exists, False otherwise.

Return type:

bool

almanack.git.find_file(*repo: Repository*, *filepath: str*, *case_insensitive: bool = False*, *extensions: list[str] = ['.md', '.txt', '.rtf', '.rst', '']*) → Object | None

Locate a file in the repository by its path.

Parameters:

- **repo** (*pygit2.Repository*) – The repository object.
- **filepath** (*str*) – The path to the file within the repository.
- **case_insensitive** (*bool*) – If True, perform case-insensitive comparison.
- **extensions** (*list[str]*) – List of possible file extensions to check (e.g., [“.md”, “”]).

Returns:

The entry of the found file, or None if no matching file is found.

Return type:

Optional[pygit2.Object]

almanack.git.get_commits(*repo: Repository*) → List[Commit]

Retrieves the list of commits from the main branch.

Parameters:

repo (*pygit2.Repository*) – The Git repository.

Returns:

List of commits in the repository.

Return type:

List[pygit2.Commit]

almanack.git.get_edited_files(*repo: Repository*, *source_commit: Commit*, *target_commit: Commit*) → List[str]

Finds all files that have been edited, added, or deleted between two specific commits.

Parameters:

- **repo** (*pygit2.Repository*) – The Git repository.
- **source_commit** (*pygit2.Commit*) – The source commit.
- **target_commit** (*pygit2.Commit*) – The target commit.

Returns:

List of file names that have been edited, added, or deleted between the two commits.

Return type:

List[str]

almanack.git.get_loc_changed(*repo_path: Path*, *source: str*, *target: str*, *file_names: List[str]*) → Dict[str, int]

Finds the total number of code lines changed for each specified file between two commits.

Parameters:

- **repo_path** (*pathlib.Path*) – The path to the git repository.
- **source** (*str*) – The source commit hash.
- **target** (*str*) – The target commit hash.
- **file_names** (*List[str]*) – List of file names to calculate changes for.

Returns:

A dictionary where the key is the filename, and the value is the lines changed (added and removed).

Return type:

Dict[str, int]

almanack.git.get_most_recent_commits(*repo_path: Path*) → tuple[str, str]

Retrieves the two most recent commit hashes in the test repositories

Parameters:

repo_path (*pathlib.Path*) – The path to the git repository.

Returns:

Tuple containing the source and target commit hashes.

Return type:

tuple[str, str]

almanack.git.get_remote_url(*repo: Repository*) → str | None

Determines the remote URL of a git repository, if available.

Parameters:

repo (*pygit2.Repository*) – The pygit2 repository object.

Returns:

The remote URL if found, otherwise None.

Return type:

Optional[str]

almanack.git.read_file(*repo: Repository*, *entry: Object | None* = *None*, *filepath: str | None* = *None*, *case_insensitive: bool* = *False*) → str | None

Read the content of a file from the repository.

Parameters:

- **repo** (*pygit2.Repository*) – The repository object.
- **entry** (*Optional[pygit2.Object]*) – The entry of the file to read. If not provided, filepath must be specified.
- **filepath** (*Optional[str]*) – The path to the file within the repository. Used if entry is not provided.
- **case_insensitive** (*bool*) – If True, perform case-insensitive comparison when using filepath.

Returns:

The content of the file as a string, or None if the file is not found or reading fails.

Return type:

Optional[str]

Reporting

This module creates entropy reports

almanack.reporting.report.pr_report(*data: Dict[str, Any]*) → str

Returns the formatted PR-specific entropy report.

Parameters:

data (*Dict[str, Any]*) – Dictionary with the entropy data.

Returns:

Formatted GitHub markdown.

Return type:

str

almanack.reporting.report.repo_report(*data: Dict[str, Any]*) → str

Returns the formatted entropy report as a string.

Parameters:

data (*Dict[str, Any]*) – Dictionary with the entropy data.

Returns:

Formatted entropy report.

Return type:

str

Metrics

Data

This module computes data for GitHub Repositories

almanack.metrics.data._get_almanack_version() → str

Seeks the current version of almanack using either pkg_resources or dunamai to determine the current version being used.

Returns:

str

A string representing the version of almanack currently being used.

almanack.metrics.data.compute_almanack_score(*almanack_table: List[Dict[str, int | float | bool]]*) → Dict[str, int | float]

Computes an Almanack score by counting boolean Almanack table metrics to provide a quick summary of software sustainability.

Parameters:

almanack_table (*List[Dict[str, Union[int, float, bool]]]*) – A list of dictionaries containing metrics. Each dictionary must have a “result” key with a value that is an int, float, or bool. A “sustainability_correlation” key is included for values to specify the relationship to sustainability: - 1 (positive correlation) - 0 (no correlation) - -1 (negative correlation)

Returns:

Dictionary of length three, including the following: 1) number of Almanack boolean metrics that passed (numerator), 2) number of total Almanack boolean metrics considered (denominator), and 3) a score that represents how likely the repository will be maintained over time based (numerator / denominator).

Return type:

Dict[str, Union[int, float]]

almanack.metrics.data.compute_pr_data(*repo_path: str, pr_branch: str, main_branch: str*) → Dict[str, Any]

Computes entropy data for a PR compared to the main branch.

Parameters:

- **repo_path** (*str*) – The local path to the Git repository.
- **pr_branch** (*str*) – The branch name for the PR.
- **main_branch** (*str*) – The branch name for the main branch.

Returns:

A dictionary containing the following key-value pairs:

- ”pr_branch”: The PR branch being analyzed.
- ”main_branch”: The main branch being compared.

- ”total_entropy_introduced”: The total entropy introduced by the PR.
- ”number_of_files_changed”: The number of files changed in the PR.
- ”entropy_per_file”: A dictionary of entropy values for each changed file.
- ”commits”: A tuple containing the most recent commits on the PR and main branches.

Return type:

dict

almanack.metrics.data.compute_repo_data(*repo_path: str*) → None

Computes comprehensive data for a GitHub repository.

Parameters:

repo_path (*str*) – The local path to the Git repository.

Returns:

A dictionary containing data key-pairs.

Return type:

dict

almanack.metrics.data.days_of_development(*repo: Repository*) → float

Parameters:

repo (*pygit2.Repository*) – Path to the git repository.

Returns:

The average number of commits per day over the period of time.

Return type:

float

almanack.metrics.data.gather_failed_almanack_metric_checks(*repo_path: str*, *ignore: List[str] | None = None*) → List[Dict[str, Any]]

Gather checks on the repository metrics and returns a list of failed checks for use in helping others understand the failed checks and rectify them.

Parameters:

- **repo_path** (*str*) – The file path to the repository which will have metrics calculated and includes boolean checks.
- **ignore** (*Optional[List[str]]*) – A list of metric IDs to ignore when running the checks. Defaults to None.

Returns:

A list of dictionaries containing the metrics and their associated results. Each dictionary includes the name, id, and

Return type:

List[Dict[str, Any]]

guidance on how to fix each failed check. The dictionary also

includes data about the almanack score for use in summarizing the results.

almanack.metrics.data.get_github_build_metrics(*repo_url: str*, *branch: str* = *'main'*, *max_runs: int* = *100*, *github_api_endpoint: str* = *'https://api.github.com/repos'*) → dict

Fetches the success ratio of the latest GitHub Actions build runs for a specified branch.

Parameters:

- **repo_url** (*str*) – The full URL of the repository (e.g., ‘ software-gardening/almanack’).
- **branch** (*str*) – The branch to filter for the workflow runs (default: “main”).
- **max_runs** (*int*) – The maximum number of latest workflow runs to analyze.
- **github_api_endpoint** (*str*) – Base API endpoint for GitHub repositories.

Returns:

The success ratio and details of the analyzed workflow runs.

Return type:

dict

almanack.metrics.data.get_table(*repo_path: str*, *ignore: List[str] | None* = *None*) → List[Dict[str, Any]]

Gather metrics on a repository and return the results in a structured format.

This function reads a metrics table from a predefined YAML file, computes relevant data from the specified repository, and associates the computed results with the metrics defined in the metrics table. If an error occurs during data computation, an exception is raised.

Parameters:

- **repo_path** (*str*) – The file path to the repository for which the Almanack runs metrics.
- **ignore** (*Optional[List[str]]*) – A list of metric IDs to ignore when running the checks. Defaults to None.

Returns:

A list of dictionaries containing the metrics and their associated results. Each dictionary includes the original metrics data along with the computed result under the key “result”.

Return type:

List[Dict[str, Any]]

Raises:

- **ReferenceError** – If there is an error encountered while processing the
- **data, providing context in the error message.** –

almanack.metrics.data.measure_coverage(*repo: Repository*, *primary_language: str | None*) → Dict[str, Any] | None

Measures code coverage for a given repository.

Parameters:

- **repo** (*pygit2.Repository*) – The pygit2 repository object to analyze.
- **primary_language** (*Optional[str]*) – The primary programming language of the repository.

Returns:

Code coverage data or an empty dictionary if unable to find code coverage data.

Return type:

Optional[dict[str,Any]]

almanack.metrics.data.parse_python_coverage_data(*repo: Repository*) → Dict[str, Any] | None

Parses coverage.py data from recognized formats such as JSON, XML, or LCOV. See here for more information: https://coverage.readthedocs.io/en/latest/cmd.html#cmd-report

Parameters:

repo (*pygit2.Repository*) – The pygit2 repository object containing code.

Returns:

A dictionary with standardized code coverage data or an empty dict if no data is found.

Return type:

Optional[Dict[str, Any]]

almanack.metrics.data.process_repo_for_analysis(*repo_url: str*) → Tuple[float | None, str | None, str | None, int | None]

Processes GitHub repository URL's to calculate entropy and other metadata. This is used to prepare data for analysis, particularly for the seedbank notebook that process PUBMED repositories.

Parameters:

repo_url (*str*) – The URL of the GitHub repository.

Returns:

A tuple containing the normalized total entropy, the date of the first commit,

the date of the most recent commit, and the total time of existence in days.

Return type:

tuple

Remote

This module focuses on remote API requests and related aspects.

almanack.metrics.remote.get_api_data(*api_endpoint: str = 'https://repos.ecosyste.ms/api/v1/repositories/lookup', params: Dict[str, str] | None = None*) → dict

Get data from an API based on the remote URL, with retry logic for GitHub rate limiting.

Parameters:

- **api_endpoint** (*str*) – The HTTP API endpoint to use for the request.
- **params** (*Optional[Dict[str, str]]*) – Additional query parameters to include in the GET request.

Returns:

The JSON response from the API as a dictionary.

Return type:

dict

Raises:

requests.RequestException – If the API call fails for reasons other than rate limiting.

Entropy

This module calculates the amount of Software information entropy

almanack.metrics.entropy.calculate_entropy.calculate_aggregate_entropy(*repo_path: Path, source_commit: Commit, target_commit: Commit, file_names: List[str]*) → float

Computes the aggregated normalized entropy score from the output of calculate_normalized_entropy for specified a Git repository. Inspired by Shannon's information theory entropy formula. We follow an approach

described by Hassan (2009) (see references).

Parameters:

- **repo_path** (*str*) – The file path to the git repository.
- **source_commit** (*pygit2.Commit*) – The git hash of the source commit.
- **target_commit** (*pygit2.Commit*) – The git hash of the target commit.
- **file_names** (*list[str]*) – List of file names to calculate entropy for.

Returns:

Normalized entropy calculation.

Return type:

float

References

- **Hassan, A. E. (2009). Predicting faults using the complexity of code changes.**

  2009 IEEE 31st International Conference on Software Engineering, 78-88.
  https://doi.org/10.1109/ICSE.2009.5070510

almanack.metrics.entropy.calculate_entropy.calculate_normalized_entropy(*repo_path: Path, source_commit: Commit, target_commit: Commit, file_names: list[str]*) → dict[str, float]

Calculates the entropy of changes in specified files between two commits, inspired by Shannon's information theory entropy formula. Normalized relative to the total lines of code changes across specified files. We follow an approach described by Hassan (2009) (see references).

Application of Entropy Calculation: Entropy measures the uncertainty in a given system. Calculating the entropy of lines of code (LoC) changed reveals the variability and complexity of modifications in each file. Higher entropy values indicate more unpredictable changes, helping identify potentially unstable code areas.

Parameters:

- **repo_path** (*str*) – The file path to the git repository.
- **source_commit** (*pygit2.Commit*) – The git hash of the source commit.
- **target_commit** (*pygit2.Commit*) – The git hash of the target commit.
- **file_names** (*list[str]*) – List of file names to calculate entropy for.

Returns:

A dictionary mapping file names to their calculated entropy.

Return type:

dict[str, float]

References

- **Hassan, A. E. (2009). Predicting faults using the complexity of code changes.**
  2009 IEEE 31st International Conference on Software Engineering, 78-88.
  https://doi.org/10.1109/ICSE.2009.5070510

This module procesess GitHub data

almanack.metrics.entropy.processing_repositories.process_pr_entropy(*repo_path: str*, *pr_branch: str*, *main_branch: str*) → str

Processes GitHub PR data to calculate a report comparing the PR branch to the main branch.

Parameters:

- **repo_path** (*str*) – The local path to the Git repository.
- **pr_branch** (*str*) – The branch name of the PR.
- **main_branch** (*str*) – The branch name for the main branch.

Returns:

A JSON string containing report data.

Return type:

str

Raises:

FileNotFoundError – If the specified directory does not contain a valid Git repository.

almanack.metrics.entropy.processing_repositories.process_repo_entropy(*repo_path: str*) → str

Processes GitHub repository data to calculate a report.

Parameters:

repo_path (*str*) – The local path to the Git repository.

Returns:

A JSON string containing the repository data and entropy metrics.

Return type:

str

Raises:

FileNotFoundError – If the specified directory does not contain a valid Git repository.

Garden Lattice

Understanding

This module focuses on the Almanack's Garden Lattice materials which encompass aspects of human understanding.

almanack.metrics.garden_lattice.understanding.includes_common_docs(*repo: Repository*) → bool

Check whether the repo includes common documentation files and directories associated with building docsites.

Parameters:

repo (*pygit2.Repository*) – The repository object.

Returns:

True if any common documentation files are found, False otherwise.

Return type:

bool

Connectedness

Module for Almanack metrics covering human connection and engagement in software projects such as contributor activity, collaboration frequency.

almanack.metrics.garden_lattice.connectedness.count_unique_contributors(*repo: Repository, since: datetime | None = None*) → int

Counts the number of unique contributors to a repository.

If a *since* datetime is provided, counts contributors who made commits after the specified datetime. Otherwise, counts all contributors.

Parameters:

- **repo** (*pygit2.Repository*) – The repository to analyze.
- **since** (*Optional[datetime]*) – The cutoff datetime. Only contributions after this datetime are counted. If None, all contributions are considered.

Returns:

The number of unique contributors.

Return type:

int

almanack.metrics.garden_lattice.connectedness.default_branch_is_not_master(*repo: Repository*) → bool

Checks if the default branch of the specified repository is “master”.

Parameters:

repo (*Repository*) – A pygit2.Repository object representing the Git repository.

Returns:

True if the default branch is “master”, False otherwise.

Return type:

bool

almanack.metrics.garden_lattice.connectedness.detect_social_media_links(*content: str*) → Dict[str, List[str]]

Analyzes README.md content to identify social media links.

Parameters:

readme_content (*str*) – The content of the README.md file as a string.

Returns:

A dictionary containing social media details discovered from readme.md content.

Return type:

Dict[str, List[str]]

almanack.metrics.garden_lattice.connectedness.find_doi_citation_data(*repo: Repository*) → Dict[str, Any]

Find and validate DOI information from a CITATION.cff file in a repository.

This function searches for a *CITATION.cff* file in the provided repository, extracts the DOI (if available), validates its format, checks its resolvability via HTTP, and performs an exact DOI lookup on the OpenAlex API.

Parameters:

repo (*pygit2.Repository*) – The repository object to search for the CITATION.cff file.

Returns:

A dictionary containing DOI-related information and metadata.

Return type:

Dict[str, Any]

almanack.metrics.garden_lattice.connectedness.is_citable(*repo: Repository*) → bool

Check if the given repository is citable.

A repository is considered citable if it contains a CITATION.cff or CITATION.bib file, or if the README.md file contains a citation section indicated by “## Citation” or “## Citing”.

Parameters:

repo (*pygit2.Repository*) – The repository to check for citation files.

Returns:

True if the repository is citable, False otherwise.

Return type:

bool

Practicality

This module focuses on the Almanack’s Garden Lattice materials which involve how people can apply software in practice.

almanack.metrics.garden_lattice.practicality.count_repo_tags(*repo: Repository*, *since: datetime | None* = *None*) → int

Counts the number of tags in a pygit2 repository.

If a *since* datetime is provided, counts only tags associated with commits made after the specified datetime. Otherwise, counts all tags in the repository.

Parameters:

- **repo** (*pygit2.Repository*) – The repository to analyze.
- **since** (*Optional[datetime]*) – The cutoff datetime. Only tags for commits after this datetime are counted. If None, all tags are counted.

Returns:

The number of tags in the repository that meet the criteria.

Return type:

int

almanack.metrics.garden_lattice.practicality.get_ecosystems_package_metrics(*repo_url: str*) → Dict[str, Any]

Fetches package data from the ecosyste.ms API and calculates metrics about the number of unique ecosystems, total version counts, and the list of ecosystem names.

Parameters:

repo_url (*str*) – The repository URL of the package to query.

Returns:

A dictionary containing information about packages related to the repository.

Return type:

Dict[str, Any]